

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



Đại Học Quốc Tế Miền Đông
Eastern International University

MARKETHUB E-COMMERCE PLATFORM

Tran Tuan Thinh – 2031200036

Tran Hai Viet – 2031200032

Software Engineering

Binh Duong, June 2025

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



Đại Học Quốc Tế Miền Đông
Eastern International University

MARKETHUB E-COMMERCE PLATFORM

Tran Tuan Thinh – 2031200036

Tran Hai Viet – 2031200032

Software Engineering

Binh Duong, June 2025

ABTRACT

This capstone project aims to develop a comprehensive e-commerce platform, named **MarketHub**, focused on facilitating the buying and selling of **Fast-Moving Consumer Goods (FMCG)** agricultural products through a group-buying model. The primary objective is to streamline the agricultural supply chain by reducing intermediary involvement and providing a transparent, efficient marketplace where sellers can list and sell their products directly to businesses and consumers.

The platform is built using modern web technologies, including **ASP.NET for the backend**, **Next.js for the frontend**, and **MySQL for the database**, ensuring a robust and scalable architecture. Key features include a **group-buying** system that allows users to pool purchases for better pricing, and integrated communication channels to enable direct interaction between sellers and buyers. Additionally, the platform offers data analytics and reporting tools that help sellers track sales performance and provide companies with insights into procurement trends.

Initial results demonstrate that **MarketHub** has successfully enhanced transaction transparency and optimized the supply chain for FMCG agricultural products. By offering an accessible, user-friendly interface, the platform enables sellers to reach a broader market while allowing businesses to benefit from direct supplier access, improved product quality, and competitive pricing.

This project underscores the potential of digital marketplaces to transform traditional agricultural trading models, especially in the **FMCG** sector. Future development will focus on expanding the platform's capabilities, including the integration of automated logistics support and enhanced group-buying functionalities.

ACKNOWLEDGEMENT

We would like to express my heartfelt gratitude to the following individuals for their invaluable contributions to my Capstone Project:

- **Our supervisor, Mr. Tran Van Tai:** For his support, insightful guidance, and encouragement throughout the development of this project, helping shape the idea and direction of the thesis.
- **Our colleagues from the Business Department:** For their innovative ideas, collaborative spirit, and invaluable assistance in refining the business logic and practical aspects of the project.

Their guidance and support have been instrumental in the successful completion of this project, and we are deeply grateful for their efforts.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES.....	v
LIST OF TABLES.....	vii
LIST OF ABBREVIATIONS.....	viii
Chapter 1. Introduction	1
1.1 The General Idea	1
1.2 Description	1
1.3 Identifying and Addressing Challenges	1
1.3.1 Our Story	1
1.3.2 Current Sales Model.....	2
1.3.3 Problems	2
1.3.4 Solutions.....	3
1.3.5 The Differences	4
1.3.6 Advantages	4
1.3.7 Disadvantages.....	5
1.4 Structure of the Report	6
Chapter 2. System Analysis and Design	7
2.1 Overview.....	7
2.2 Requirements Analysis.....	7
2.2.1 Functional Requirements	7
2.2.2 Non-Functional Requirements.....	7
2.3 Use Case Diagram.....	7
2.4 Activity Diagram.....	9
2.5 Class Diagram	11
2.6 Entity Relationship Diagram (ERD).....	11
2.7 System Design	13
Chapter 3. Implementation.....	14
3.1 Overview.....	14
3.2 Development Environment and Tools	14
3.2.1 Front-end Technologies.....	14
3.2.2 Back-end Technologies	15
3.2.3 Database Management	15
3.2.4 Additional Technologies.....	15

3.3 Business Logic	16
3.3.1 Discount Types (Automatic Discount)	16
3.3.2 Aggregate Discount Calculation	16
3.3.3 Practical Application	17
3.4 Back-end Development.....	17
3.4.1 Triggers.....	17
3.4.2 API Gateways.....	19
3.4.3 Token	25
3.4.4 Security Measures.....	25
3.4.5 Data Transfer Object (DTO)	26
3.4.6 Email Service.....	27
3.4.7 Seed Data	27
3.4.8 Docker.....	28
3.5 Front-end Development	29
3.5.1 Banner.....	30
3.5.2 Auth Page.....	30
3.5.3 Contact Page.....	31
3.5.4 Home Page.....	32
3.5.5 Profile.....	33
3.5.6 Product Page	36
3.5.7 Cart Page	37
3.5.8 Additional Features	37
Chapter 4. Conclusion.....	39
4.1 Overall Project Summary	39
4.2 Technological and Methodological Impact	39
4.3 Future Improvements and Scalability	39
4.4 Business and Societal Impact.....	39
Chapter 5. Reflection and Future Enhancements	41
5.1 Overview.....	41
5.2 Key Learnings	41
5.3 Challenge Faced.....	41
5.4 Future Enhancements	42
Chapter 6. References	44

LIST OF FIGURES

Figure 2.1: Client Use Case.....	8
Figure 2.2: Sellers and Buyers Use Case	8
Figure 2.3: Overall System Activity Diagram.....	9
Figure 2.4: Manage Cart Activity Diagram	9
Figure 2.5: Manage Product Activity Diagram	10
Figure 2.6: Manage Personal Information Activity Diagram	10
Figure 2.7: Sign-In Activity Diagram	11
Figure 2.8: Class Diagram	11
Figure 2.9: Entity Relationships Diagram.....	12
Figure 2.10: System Design.....	13
Figure 3.1: Backend System	16
Figure 3.2: Advanced Data Query	20
Figure 3.3: Address API.....	20
Figure 3.4: Auth API.....	21
Figure 3.5: Discounts API.....	21
Figure 3.6: Contact API.....	22
Figure 3.7: Users API	22
Figure 3.8: Orders API	23
Figure 3.9: Products API	23
Figure 3.10: Product_Main_Categories API.....	24
Figure 3.11: Product_Categories API.....	24
Figure 3.12: Feedbacks API.....	25
Figure 3.13: Conversation API.....	25
Figure 3.14: Get Request (DTO)	26
Figure 3.15: Post/ Patch Request (DTO)	26
Figure 3.16: Docker Architecture	28
Figure 3.17: Containerize with Docker	28
Figure 3.18: Publish to DockerHub	29
Figure 3.19: DockerHub Published Images	29
Figure 3.20: Banner.....	30
Figure 3.21: Sign In	30
Figure 3.22: Sign Up.....	30

Figure 3.23: Recovery.....	31
Figure 3.24:Contact.....	31
Figure 3.25: Home	32
Figure 3.26: Home (with View More Button)	32
Figure 3.27: Home (Buy Together)	32
Figure 3.28: Personal Information.....	33
Figure 3.29: Reset Password.....	33
Figure 3.30: Chart/ Map	33
Figure 3.31: Province Detail	34
Figure 3.32: Personal History Order	34
Figure 3.33: Feedback Order	35
Figure 3.34: Personal Product for Sell	35
Figure 3.35: Add/ Edit Product.....	35
Figure 3.36: Product.....	36
Figure 3.37: Product (with Pagination)	36
Figure 3.38: Product Detail.....	36
Figure 3.39: Cart	37
Figure 3.40: Group Buy	37
Figure 3.41: Top-up Button	37
Figure 3.42: Search Product Bar.....	37
Figure 3.43: ChatBox	38

LIST OF TABLES

Table 2.1: Explanations of Entity Table.....	12
Table 2.2: Entity of Systems	12

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ASP.NET	Active Server Pages .NET
B2C	Business to Consumer
C2C	Customer to Customer
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DSL	Domain-Specific Language
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
F&B	Food and Beverage
FK	Foreign Key
FMCG	Fast-Moving Consumer Goods
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
M-a-P Shop	Mom and Pop Shop
MSSQL	Microsoft SQL
ORM	Object-Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
SDGs	United Nations Sustainable Development Goals
SQL	Structured Query Language
2FA	Two-Factor Authentication

Chapter 1. Introduction

1.1 The General Idea

This capstone project focuses on developing an agricultural B2C (Business to Customer) e-commerce platform that facilitates the buying and selling of FMCG (Fast-Moving Consumer Goods) through a group-buying model.

1.2 Description

The e-commerce platform developed in this project focuses on connecting distributors, who are sellers, of FMCG agricultural products directly with M-a-P Shops (Mom-and-Pop Shops), main buyers, through a group-buying model. It serves as a digital marketplace where distributors can list products like grains, vegetables, and other FMCG items. Buyers can browse these listings and join group purchases to secure larger quantities at lower prices, leveraging the power of collective buying.

By simplifying the transaction process and reducing the need for intermediaries (such as speculators), the platform ensures fairer pricing for factories while lowering costs for buyers. Key features include group-buy feature, secure payment processing, product quality verification, and order tracking, enhancing both user experience and transaction security.

This project aims to foster a sustainable, community-driven ecosystem that strengthens the agricultural supply chain, promotes direct trade, and provides transparency and reliability for all stakeholders involved.

1.3 Identifying and Addressing Challenges

1.3.1 Our Story

During a routine observation in Binh Duong, a troubling pattern emerged: the prices of FMCG agricultural products were significantly higher than expected, yet the quality often failed to justify the elevated costs. This phenomenon extended beyond Binh Duong, as similar issues were identified in surrounding provinces, where consumers across both urban and rural areas faced a persistent mismatch between product pricing and quality.

Driven by curiosity regarding this trend, a hypothesis was formed that agricultural products might be more affordable in rural areas than in urban centers. However, uncertainty remained about whether this applied to FMCG agricultural products specifically. To gain a clearer understanding of the underlying market discrepancies, a field study was initiated in Soc Trang, a province in Vietnam's Mekong Delta, with the aim of exploring the rural agricultural supply chain and investigating whether regional differences could account for the inconsistencies observed in Binh Duong.

Findings from Soc Trang proved both unexpected and concerning. Although prices of FMCG agricultural products in rural areas were indeed lower than those in urban centers such as Binh Duong, product quality often failed to meet even basic standards. Infrastructural limitations and inadequate communication channels in rural regions were identified as key contributors to these shortcomings. The price disparity, it became evident, was not attributed to reduced transportation costs or fewer intermediaries, but rather to a lack of quality control and inefficient distribution systems.

This misalignment between price and quality underscored more than a minor inconvenience; it exposed fundamental inefficiencies within the agricultural supply chain. The field study offered critical insights into the structural causes of these issues, revealing that the root of the problem extended beyond regional pricing differences. Instead, the situation reflected a broader set of challenges involving fragmented logistics, limited transparency, and poor coordination among producers, distributors, and retailers.

The outcome of this investigation pointed to a compelling need for systemic improvement. Potential solutions began to take shape - ones aimed at narrowing the gap between urban and rural markets, elevating product quality, and establishing fair pricing mechanisms. By optimizing distribution networks and implementing more robust communication infrastructures, a more efficient and equitable market environment for FMCG agricultural products could be envisioned, ultimately benefiting all stakeholders across the supply chain.

1.3.2 Current Sales Model

After researching, our team conclude the Current Model of Selling FMCG for Agricultural Products as below:

Factory → Distributors → Speculators → M-a-P Shops → Customers

Explanation: Factories produce FMCG agricultural products, which are then purchased by distributors with little to no detailed evaluation. Following this, speculators acquire products from one or multiple distributors, also with minimal consideration of quality or demand. Subsequently, these speculators supply the products to M-a-P Shops, often without assessing whether the shops genuinely need the inventory. Finally, customers visit these M-a-P Shops to purchase FMCG agricultural products, often without alternatives or insights into the product's origin or quality.

1.3.3 Problems

Driven by the need for further insights, the focus was shifted to rural areas to determine whether similar issues—such as communication gaps between sellers and buyers—were also present. In

Soc Trang, the situation appeared to be even more challenging. The investigation revealed several key problems, including the following:

- **Problem 1:** M-a-P Shops are unaware of the distributor sources for their products, forcing them to rely on speculators. As a result, speculators tend to purchase smaller quantities from distributors, leading to higher prices due to transportation costs. Larger orders would reduce prices, but smaller orders result in higher costs. Moreover, the lack of product specifications made it easier for sellers to inflate prices, while M-a-P Shops were more likely to receive counterfeit or substandard goods.
- **Problem 2:** In the current process, speculators lack knowledge of the specific needs of M-a-P Shops, leading them to deliver excessive quantities of products to sell. This inefficiency increases transportation costs and contributes to wastage.
- **Problem 3:** Distributors lack an efficient inventory management system and cannot update M-a-P Shops' demand in real-time. This leads to overstocking, as distributors purchase large quantities without knowing the specific needs of M-a-P Shops. As a result, distributors face higher inventory costs, including storage, staffing, and potential product expiration, causing negative cash flow.

1.3.4 Solutions

Through this investigation, we explored an innovative group-buying model inspired by "Super Indonesia," which has proven successful in addressing similar issues. This model effectively tackles key problems, such as logistics inefficiencies and the lack of direct communication between suppliers and buyers.

Solution Model of Selling FMCG for Agricultural Products:

Factory → Distributors → MarketHub (WebApp) → M-a-P Shops → Customers

Explanation: Factories produce FMCG agricultural products, which are then purchased by distributors who evaluate the demand from M-a-P Shops. These distributors subsequently supply the products directly to M-a-P Shops through its delivery team. Finally, customers visit these M-a-P Shops to purchase FMCG agricultural products, which have insight about the product's origin and quality.

By adapting and applying this approach—skipping speculators and utilizing a web app—the following outcomes could be achieved:

- **Result 1:** Distributors can directly understand the specific products needed by M-a-P Shops. This reduces transportation costs and ensures that the products ordered align with demand, ultimately lowering product prices for M-a-P Shops and reducing prices for consumers.

- **Result 2:** Distributors can accurately forecast the products they need to import into their inventories. This reduces unnecessary utilities cost (such as warehouse space usage, management fees, etc) leading to more efficient operations and reduced costs.
- **Result 3:** Implementing a group-buying feature for M-a-P Shops enhances their purchasing power by allowing them to pool their orders. This results in bulk purchasing, leading to lower unit costs due to economies of scale. By ordering larger quantities, M-a-P Shops can take advantage of reduced utility and overhead costs, further lowering the final product prices.

This research journey not only deepened the understanding of the market but also inspired the development of a solution tailored to Vietnam's unique context, aiming to foster a fairer and more efficient FMCG agricultural supply chain.

1.3.5 The Differences

This project specifically targets the need for affordable FMCG agricultural products in Vietnam. To achieve this, the platform introduces a "GroupBuy" feature, allowing users to pool their orders for bulk purchasing, which unlocks significant discounts typically reserved for larger buyers.

Unlike many existing platforms that primarily serve individual purchases at standard prices, this platform empowers users to leverage collective buying power, making FMCG agricultural products more accessible and affordable.

For instance, Amazon, a global and B2C e-commerce platform, also sells agricultural products but does not primarily focus on FMCG items and lacks a "GroupBuy" feature to reduce product prices. In Vietnam, C2C (Consumer to Consumer) platforms like Lazada, Shopee, and Tiki also operate as e-commerce platforms but do not cater to FMCG agricultural products. While the "GroupBuy" feature is used in the F&B (Food and Beverage) industry on platforms such as GrabFood or ShopeeFood, it is not yet applied in the FMCG agricultural sector, making this project unique in its approach.

1.3.6 Advantages

Conceptual Advantages

- **Addressing Market Demand:** The platform meets the growing need for a reliable and efficient online marketplace for fast-moving consumer goods and agricultural products, especially in rural and semi-urban areas where such services are limited.
- **Supporting Local Farmers and Producers:** By offering a direct channel between producers and consumers, the system empowers local farmers and small-scale suppliers to

reach broader markets without relying heavily on intermediaries, improving their income and market presence.

- **Enhancing Transparency and Trust:** Features such as detailed order tracking, clear payment deadlines, and automated notifications contribute to a transparent user experience, fostering trust between buyers and sellers.
- **Promoting Bulk Purchasing and Discounts:** The platform encourages group-buying models, enabling users to purchase in bulk and benefit from collective discounts. This not only reduces costs for consumers but also increases the sales volume for sellers.

Technical Advantages

- **Secure and Convenient User Authentication:** The system implements secure authentication flows using JWT (Json Web Token) and supports third-party login through Google OAuth 2.0, providing both security and user convenience.
- **Interactive Data Visualization:** Integrated charts and maps offer visual insights into product trends, order statistics, and geographical distribution, enhancing the admin's ability to analyze and make informed decisions.
- **Stable and Scalable Architecture:** Built using a robust stack (Next.js for frontend, ASP.NET Core for backend, and MySQL for database), the architecture supports scalability and maintainability, making it suitable for future expansion.
- **Real-Time Feedback System:** Users can submit feedback directly through the platform, enabling real-time communication with the admin and helping to improve service quality.
- **Modern Technology Stack:** The use of modern technologies such as Docker, RESTful APIs, and Entity Framework Core ensures performance, modularity, and ease of development and deployment.

1.3.7 Disadvantages

Conceptual Limitations

- **Market Competition:** Entering the e-commerce space means facing strong competition from well-established platforms that already have brand recognition, logistics infrastructure, and a large customer base.
- **Dependence on Internet Access:** The effectiveness of the system relies heavily on users having stable internet access, which may be limited or inconsistent in remote or underdeveloped areas.
- **Logistics Complexity:** Managing delivery, especially for perishable agricultural products, can be challenging. Ensuring timely and safe transportation requires coordination that may be beyond the platform's control in its early stages.

Technical Limitations

- **Security Risks:** Despite using standard security practices like JWT and HTTPS, the system remains vulnerable to common threats such as phishing, SQL injection, or unauthorized access if not regularly updated and monitored.
- **Scalability Constraints:** While the current architecture supports basic scalability, handling a sudden increase in user base or concurrent orders may require further infrastructure optimization and resource scaling.
- **Integration Challenges:** Integrating third-party services such as MoMo payment, email services, and mapping APIs can introduce complexity and maintenance overhead, especially when external APIs change or become unstable.
- **Initial Development Time:** Building a full-featured, end-to-end platform from scratch requires a significant amount of time for planning, coding, testing, and debugging, which may delay deployment or limit features during the early phase.

1.4 Structure of the Report

This report is structured to provide a detailed of MarketHub project. Following this introduction, Chapter 2 will analyze and design the architecture of software. Chapter 3 will show how much I implemented. Chapter 4 will reflect on the overall learning experience, discussing the skills acquired and their relevance to my future career. Finally, Chapter 5 will provide a conclusion, summarizing the key takeaways and their impact on my professional growth.

Chapter 2. System Analysis and Design

2.1 Overview

Describe the primary goals of the system, such as connecting sellers and buyers of FMCG agricultural products through a group-buying model and offering affordable pricing.

Identify the problems the system addresses, like reducing purchasing costs and optimizing the supply chain for agricultural products.

2.2 Requirements Analysis

2.2.1 Functional Requirements

- **User Authentication:** Users must register and log in to access the platform, ensuring secure access and personalized services. This includes features such as password recovery, account verification, and role-based access control (e.g., distinguishing between buyers and sellers).
- **Group-Buy feature for users:** Allows multiple users to combine their orders to access discounts on bulk purchases.
- **Product Search and browsing functionality:** Users can search for and browse products by category, price, or other relevant filters.
- **Order tracking and delivery status updates:** Users can track the status of their orders and receive updates on delivery progress.
- **Product managing for sellers:** Sellers can easily add, edit, and manage their product listings, including setting prices, stock levels, and promotional offers.

2.2.2 Non-Functional Requirements

- **Security:** Secure user authentication and data encryption, protecting sensitive user information and transaction data.
- **Usability:** The platform should have an intuitive, user-friendly interface, ensuring that users, both buyers and sellers, can easily navigate and perform tasks.
- **Maintainability:** The system should be designed to allow seamless integration of new features.

2.3 Use Case Diagram

The Use Case Diagram provides a visual representation of the key interactions between users and the system, outlining the various functionalities and how users will engage with the platform to achieve their goals.

Notation:

- Manage including CRUD: Create, Read, Update, Delete

- 2 Types of Clients: Sellers & Buyers

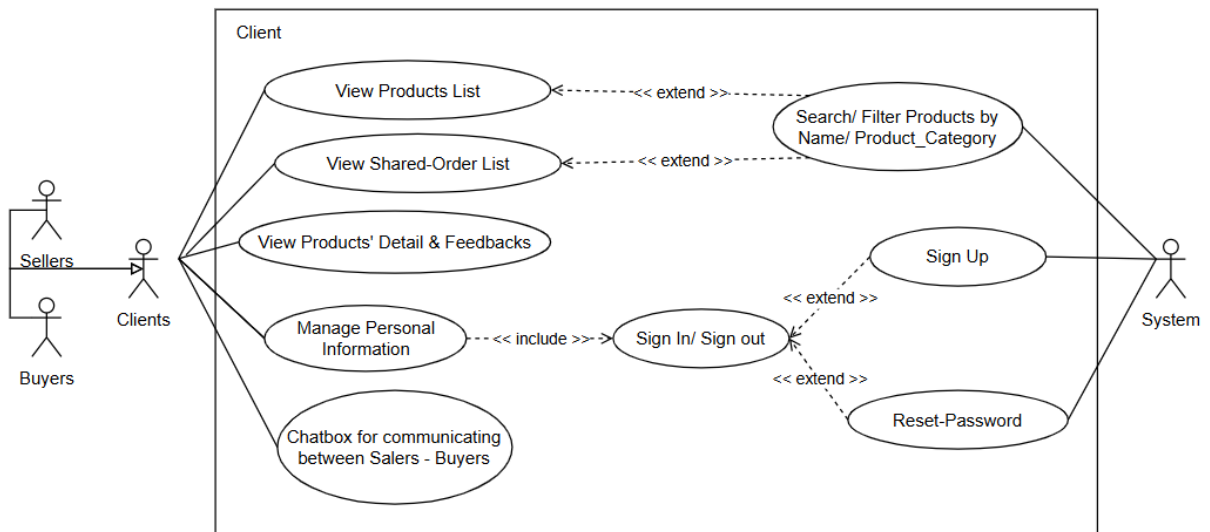


Figure 2.1: Client Use Case

Notation:

- Manage including CRUD: Create, Read, Update, Delete

- 2 Types of Clients: Sellers & Buyers

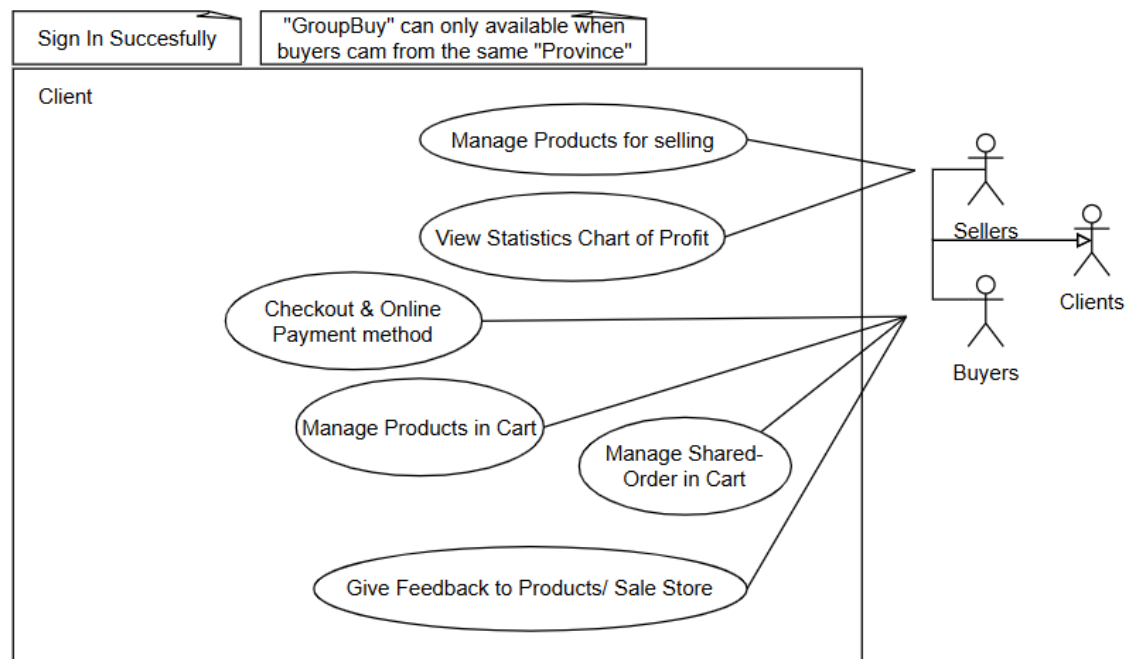


Figure 2.2: Sellers and Buyers Use Case

2.4 Activity Diagram

The Activity Diagram outlines the step-by-step workflow of specific processes within the system, showing the sequence of actions and decisions made during user interactions.

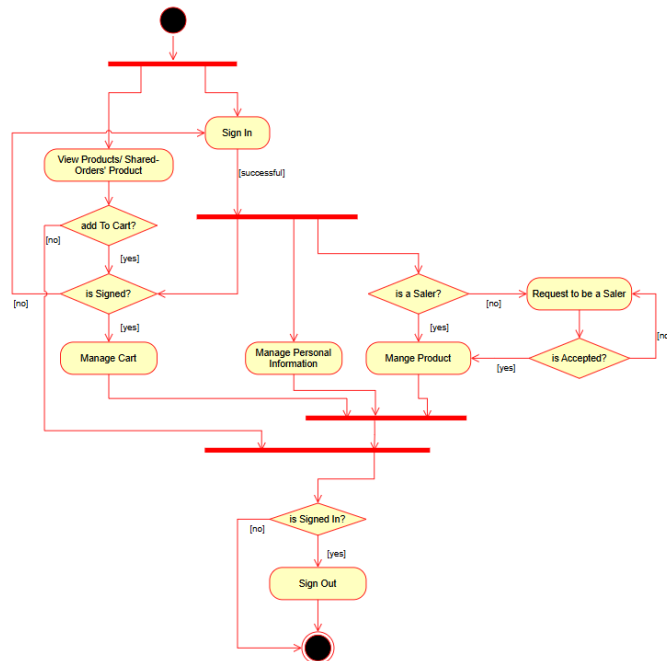


Figure 2.3: Overall System Activity Diagram

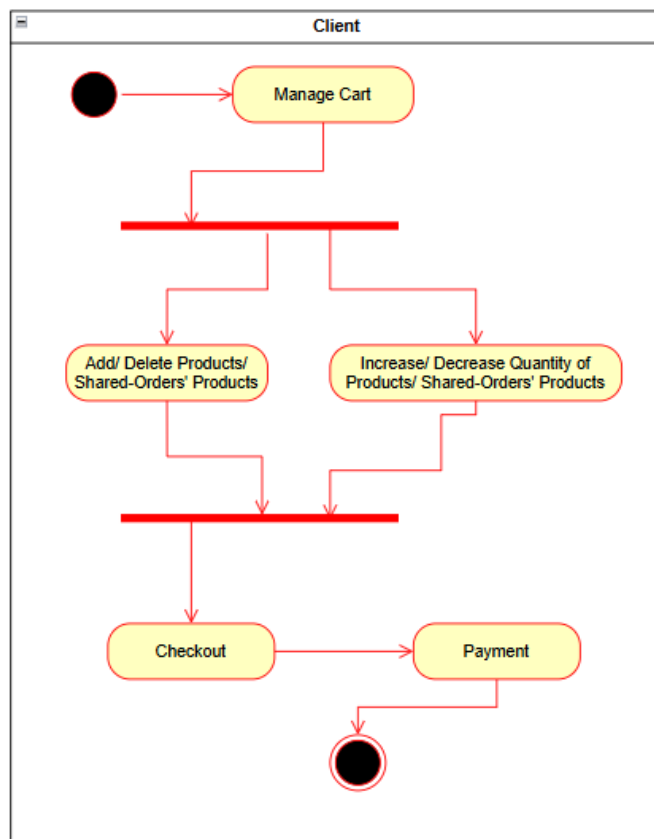


Figure 2.4: Manage Cart Activity Diagram

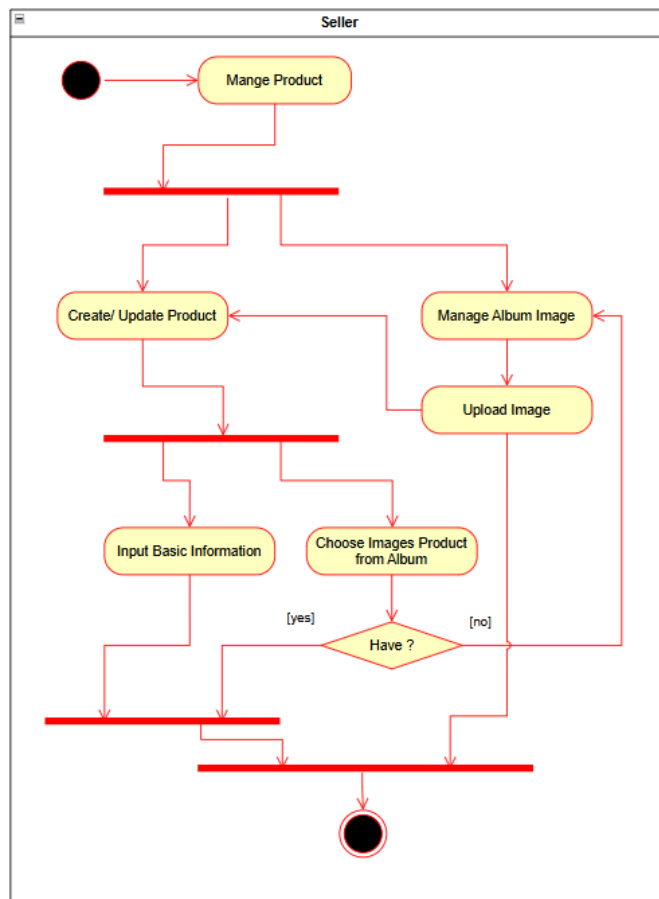


Figure 2.5: Manage Product Activity Diagram

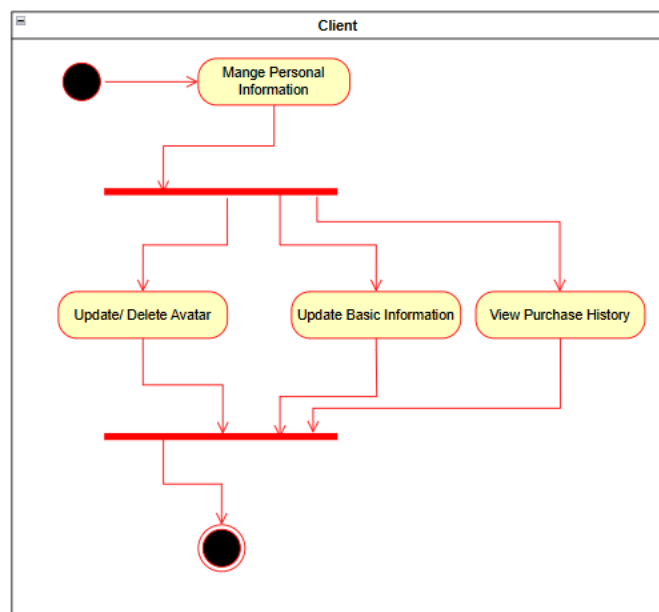


Figure 2.6: Manage Personal Information Activity Diagram

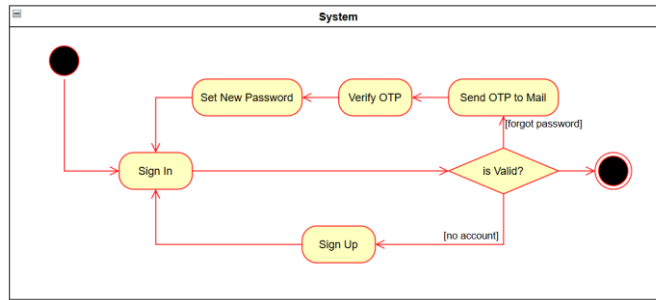


Figure 2.7: Sign-In Activity Diagram

2.5 Class Diagram

The Class Diagram shows the system's static structure, detailing classes, their attributes, methods, and relationships, serving as a blueprint for object-oriented design.

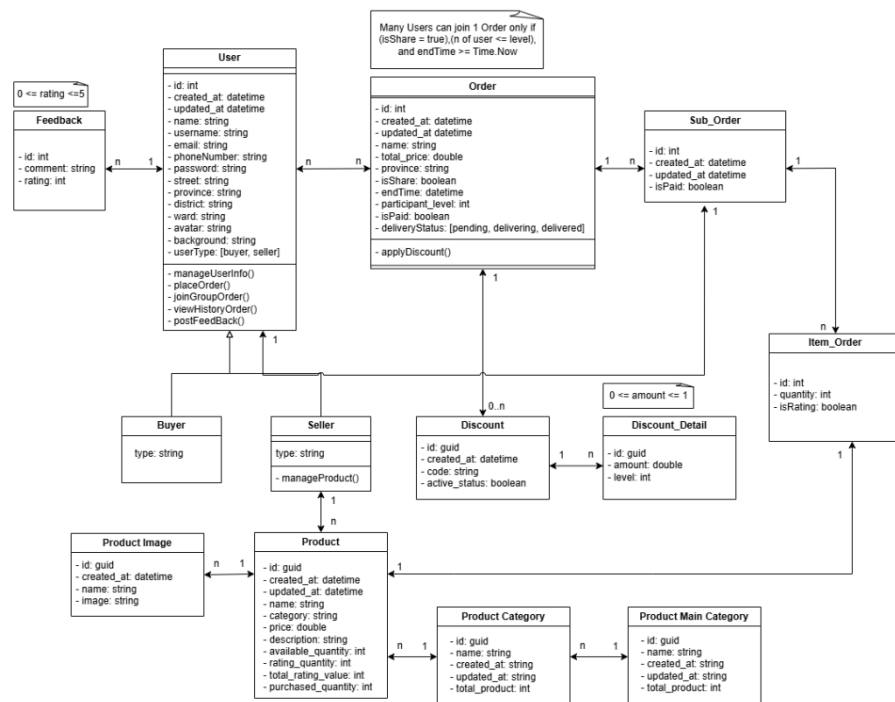


Figure 2.8: Class Diagram

2.6 Entity Relationship Diagram (ERD)

The ERD (Entity Relationship Diagram) visually represents the system's data structure by illustrating entities, their attributes, and the relationships between them. It defines **PK (Primary Key)** as unique identifiers for records and uses **FK (Foreign Key)** to establish connections between tables, ensuring referential integrity. By mapping **one-to-one**, **one-to-many**, and **many-to-many** relationships, the ERD aids in designing an efficient and well-structured database.

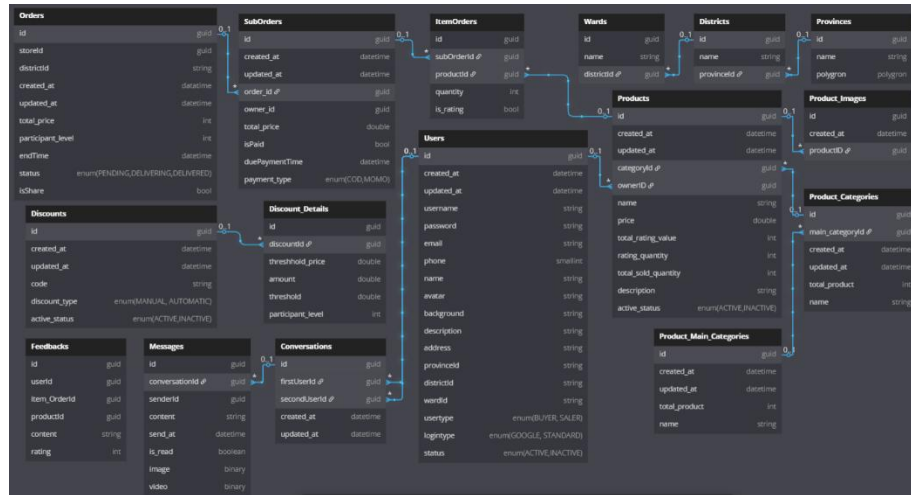


Figure 2.9: Entity Relationships Diagram

**** Notes:**

Table 2.1: Explanations of Entity Table

Symbol	Meanings
TempId ref: > Example.id	<i>TempId</i> is a FK in a many-to-one relationship with the FK of <i>Example</i> table.

Table 2.2: Entity of Systems

Entity	PK	FK	Relationship
Users	Id: Guid		
Products	Id: Guid	CategoryId: Guid	ref: > Product_Categories.id
		OwnerId: Guid	ref: > Users.id
Product_Images	Id: Guid	ProductId: Guid	ref: > Products.id
Product_Main_Categories	Id: Guid		
Product_Categories	Id: Guid	Main_CategoryId: Guid	ref: > Product_Main_Categories.id
Orders	Id: Guid		
Sub_Orders	Id: Guid	OrderId: Guid	ref: > Orders.id
		OwnerId: Guid	ref: > Users.id
Item_Orders	Id: Guid	Sub_OrderId: Guid	ref: > SubOrders.id
		ProductId: Guid	ref: > Products.id
Discounts	Id: Guid		
Discount_Details	Id: Guid	DistrictId: Guid	ref: > Discounts.id
Conversations	Id: Guid	FirstUserId: Guid	ref: > Users.id
		SecondUserId: Guid	ref: > Users.id
Messages	Id: Guid	ConversationId: Guid	ref: > Conversations.id
Feedbacks	Id: Guid		
Provinces	Id: String		
Districts	Id: String	ProvinceId: String	ref: > Provinces.id
Wards	Id: String	DistrictId: String	ref: > District.id

2.7 System Design

The architecture of the system follows the **Microservice Architecture**. The Microservice Architecture enables the e-commerce platform to be highly scalable, maintainable, and fault-tolerant. Each service is independent, making it easier to deploy, update, and manage. While this architecture introduces complexity, its advantages in scalability, flexibility, and fault isolation make it a powerful approach for modern applications.

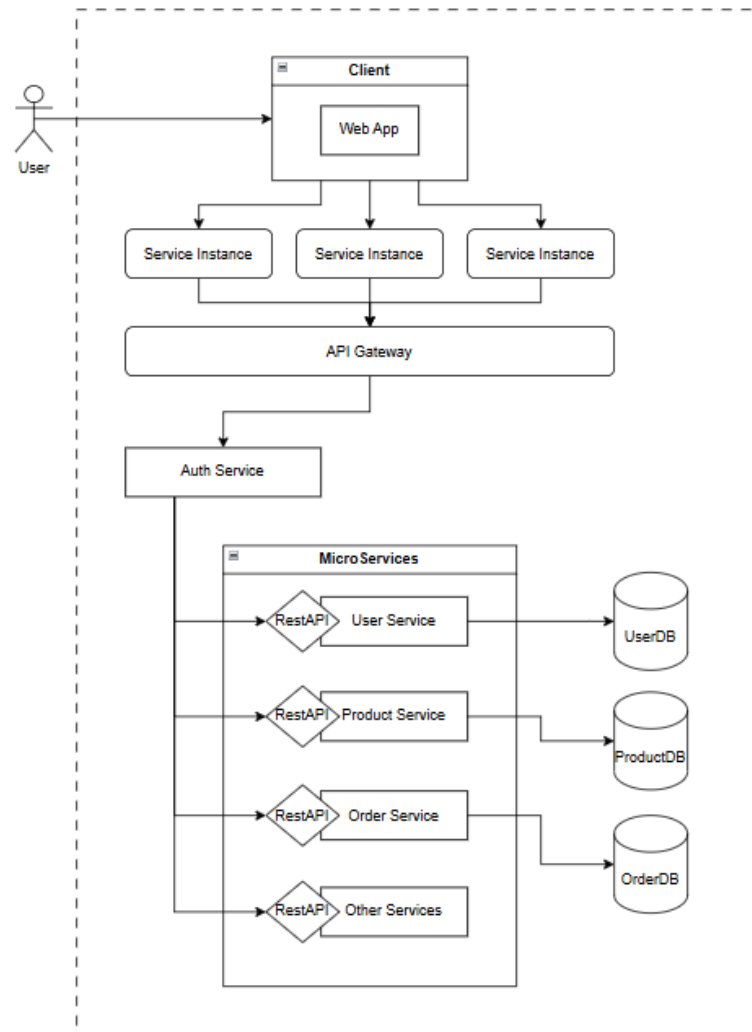


Figure 2.10: System Design

Chapter 3. Implementation

3.1 Overview

This chapter outlines the development process of the e-commerce platform, highlighting the key features, functionalities, and technologies utilized during the implementation phase. It provides an in-depth look at how the system was built, the challenges encountered, and the solutions applied to achieve a robust, scalable, and user-friendly platform.

3.2 Development Environment and Tools

During this project, various technologies were utilized to ensure a robust, efficient, and scalable solution. Below is an overview of the key technologies employed:

3.2.1 Front-end Technologies

- **HTML5:** Used for structuring the web content.
- **CSS3:** Applied for styling the web pages.
- **JavaScript:** The core programming language for making the web pages interactive. You might also mention frameworks like React.js or Next.js if used in your project.
- **Next.js:** A React-based framework used to build server-side rendered (SSR) and statically generated web pages. Next.js improves performance, and simplifies routing with its file-based structure.
- **HeroUI:** A modern React UI library that enables developers to build accessible and customizable web applications with ease. It offers a comprehensive set of components, all built on top of Tailwind CSS and React Aria. This combination ensures that HeroUI provides both aesthetic appeal and robust functionality for React applications.
- **Tailwind CSS:** A utility-first CSS framework that allows for the rapid creation of custom designs. It provides a collection of pre-built utility classes.
- **Redux:** A state management library for JavaScript applications, typically used with React. Redux helps manage global application state in a predictable and maintainable way. It allows for centralized storage of application data, making it easier to debug and share state across components without prop-drilling.
- **Lodash:** A JavaScript utility library that provides helpful functions for working with arrays, objects, and other data types. It simplifies tasks like data manipulation, deep cloning, and function throttling, improving code readability and performance.
- **Moment:** A JavaScript library used for parsing, validating, manipulating, and formatting dates and times. It helps developers manage time-related tasks efficiently, especially when handling different time zones or complex date calculations.

- **Sonner:** A React toast component designed to provide notifications in a user-friendly and customizable way.

3.2.2 Back-end Technologies

- **C#:** An object-oriented programming language used to develop back-end logic in ASP.NET Core applications.
- **ASP.NET Core:** A high-performance, cross-platform framework for building web applications, APIs, and microservices.
- **Entity Framework Core (EF Core):** An ORM (Object-Relational Mapping) framework that simplifies database interactions. EF Core allows developers to work with data as strongly typed objects, abstracting away SQL queries. It supports features like LINQ queries, change tracking, migrations, and database seeding.
- **JWT (JSON Web Token):** Used for stateless authentication in the application. JWTs are securely transmitted between the client and server for user authentication, ensuring that only authorized users can access specific resources.

3.2.3 Database Management

Structured Query Language (SQL) is a standardized programming language used to manage and manipulate relational databases. SQL allows users to query, update, and manage data in databases.

- **MySQL:** An open-source RDBMS (Relational Database Management System) that is widely used for web applications. It is based on the client-server model and is known for its speed, reliability, and ease of use.

*=> The decision to use **MySQL** instead of **MSSQL (Microsoft SQL)** is primarily due to the fact that this project is intended for demonstration purposes. Furthermore, **MySQL** is lighter than **MSSQL**, making it more suitable for running on personal laptops.*

3.2.4 Additional Technologies

- **Version Control:** Git and Github for code management. It ensured version consistency, streamlined merging, and provided backup and recovery through version history.
- **Docker:** Docker was utilized to containerize the application, ensuring consistent development, testing, and production environments. Docker containers provide portability and scalability, making it easier to deploy the application in different environments.
- **Swagger:** Integrated with ASP.NET Core to automatically generate API documentation. Swagger provides an interactive UI for testing API endpoints and helps ensure that the API is well-documented and easy to consume.

- **Postman:** Used to test the back-end services and ensure that the endpoints work as expected. These tools allowed us to simulate various HTTP requests, check response statuses, and validate that the data returned by the API is correct.
- **DbDiagram.io:** A free, user-friendly online tool designed for developers and data analysts to create Entity-Relationship Diagram (ERD) by writing code. It allows users to define database schemas using a simple Domain-Specific Language (DSL), facilitating quick and efficient diagram creation.
- **Draw.io:** Web-based diagramming tool that allows to create flowcharts, process diagrams, UML diagrams, network diagrams, and more. It is widely used for visualizing systems, processes, data models, and various other concepts in both personal and professional environments.

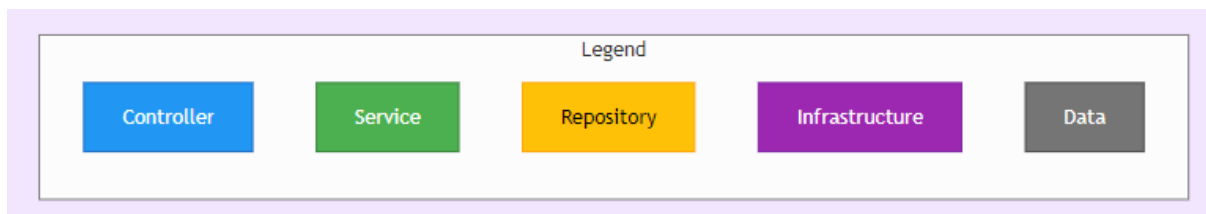


Figure 3.1: Backend System

3.3 Business Logic

The discount calculation logic is designed to determine the optimal pricing strategy based on different discount schemes. The system considers multiple discounts types and applies them in a structured manner to ensure customers receive the best possible pricing while maintaining business profitability.

3.3.1 Discount Types (Automatic Discount)

- **Volume-Based Discount:** This discount is applied based on the quantity or value of an individual order. Higher order values or quantities qualify for greater discounts, incentivizing larger purchases.
- **Group-Buy Discount:** This discount is applied when multiple buyers participate in a group order. The discount level depends on the number of participants, encouraging collective purchases to benefit from reduced prices.

3.3.2 Aggregate Discount Calculation

The total order value is calculated first, combining all individual orders into a single amount. The volume-based discount is then applied to the total, followed by the group discount. This approach optimizes discounts by leveraging the combined order value.

3.3.3 Practical Application

Scenario: Applying Volume-Based and Group-Buy Discounts

More Information:

- Volume-Based Discount: (Threshold – Discount Percentage)
 - 200 units – 5%
 - 250 units – 7%
 - 500 units – 10%
- Group-Buy Discount: (Participant Level– Discount Percentage)
 - 1 – 10%
 - 2 – 15%
 - 3 – 20%
 - 4 – 25%
 - 5 – 40%

Consider a case where two customers place separate orders:

- **Order A:** 200 units
- **Order B:** 50 units

Apply Aggregate Discount Calculation:

- Combine both orders: **Total 250 units and Participant Level 1**
- Apply volume-based discount based on total quantity: **250 units**
7% discount $\rightarrow 250 * (1 - 0.07) = 232.5$ units
- Apply group-buy discount based on participant level: **1**
10% discount $\rightarrow 232.5 * (1 - 0.1) = 209.25$ units

3.4 Back-end Development

3.4.1 Triggers

In a product and order management system, maintaining data integrity is crucial. To ensure that database tables are consistently updated, we use **Triggers** in MySQL.

**** AFTER INSERT** triggers are used to update relevant information immediately after new data is inserted.

After Product Insert Trigger:

- **Purpose:** After a new product is added, update the total product count in the corresponding product category.
- **Trigger Logic:**
 - Retrieve the **Quantity** of the newly inserted product.

- Increase the **Total_Product** field in the **product_categories table** by the inserted product's quantity.

After Product Categories Insert Trigger:

- **Purpose:** After a new product category is inserted, update the total product count in the main product category.
- **Trigger Logic:**
 - Retrieve the **Total_Product** value of the newly inserted category.
 - Increase the **Total_Product** field in the **product_main_categories table** by this value.

After Feedbacks Insert Trigger:

- **Purpose:** After a new feedback entry is inserted, update product rating statistics and mark the corresponding item order as rated.
- **Trigger Logic:**
 - Increase the **Total_Rating_Quantity** and **Total_Rating_Value** fields in the products table.
 - Set the **IsRating** field to true in the **item_orders table**.

**** AFTER UPDATE** triggers are used to ensure that related data is adjusted when changes are made to an existing record.

After Product Update Trigger:

- **Purpose:** Update the total product count in the **product_categories table** if the product quantity changes.
- **Trigger Logic:**
 - Calculate the difference between the new and old quantity values.
 - Adjust the **Total_Product** field in the **product_categories table** accordingly.

After Product Categories Update Trigger:

- **Purpose:** Update the total product count in the **product_main_categories table** if the total product count in a category changes.
- **Trigger Logic:**
 - Compute the difference between the new and old **Total_Product** values.
 - Adjust the **Total_Product** field in the **product_main_categories table**.

After Feedbacks Update Trigger:

- **Purpose:** Update product rating statistics if a feedback rating is modified.
- **Trigger Logic:**

- Update product rating statistics if a feedback rating is modified.
- Adjust the **Total_Rating_Value** field in the **products table**.

=> The implementation of **AFTER INSERT** and **AFTER UPDATE** triggers in the database ensures automatic data consistency and reduces the need for manual updates in the application layer. This helps maintain accurate records for product inventory, order pricing, and customer feedback management. By using triggers, the system achieves better performance and reliability while ensuring data integrity across different tables.

3.4.2 API Gateways

The API is designed based on **RESTful principles**, ensuring stateless communication, scalability, and maintainability. By adhering to these principles, the API allows for efficient interaction between the frontend and backend, promoting flexibility and ease of integration. It provides a standardized interface, where various **HTTP** methods such as **GET**, **POST**, **PUT**, **PATCH**, and **DELETE** are utilized to perform **CRUD** (Create, Read, Update, Delete) operations on resources.

Additionally, the API supports **advanced data querying capabilities**, including:

- **Field Selection (SelectFields)** – Clients can specify which fields to retrieve, reducing payload size and optimizing performance.
- **Filtering (Filter)** – Data can be filtered based on specific criteria, allowing more precise retrieval of relevant records.
- **Sorting (SortBy, SortOrder)** – Results can be sorted by any field in ascending or descending order to match business requirements.
- **Pagination (Page, PageSize)** – Large datasets are broken down into smaller chunks, improving performance and user experience.

Here are some key APIs implemented in the project:

Name	Description
SelectFields string (query)	SelectFields
Filter string (query)	Filter
SortBy string (query)	SortBy
SortOrder string (query)	SortOrder
Page integer(\$int32) (query)	Page
PageSize integer(\$int32) (query)	PageSize

Figure 3.2: Advanced Data Query

Address	
GET	/api/address/provinces
GET	/api/address/provinces/{provinceId}
GET	/api/address/districts/{provinceId}
GET	/api/address/wards/{districtId}

Figure 3.3: Address API

Auth

POST /api/auth/signup

POST /api/auth/signin

GET /api/auth/google

POST /api/auth/send-otp

POST /api/auth/reset-password

Figure 3.4: Auth API

Discounts

GET /api/discounts

GET /api/discounts/{id}

PATCH /api/discounts/{id}

DELETE /api/discounts/{id}

Figure 3.5: Discounts API

Contact	
POST	/api/contact/send-contact

Figure 3.6:Contact API

Users	
GET	/api/users
PATCH	/api/users
DELETE	/api/users
GET	/api/users/{id}
POST	/api/users/change-password
GET	/api/users/orders
GET	/api/users/item-orders/{subOrderId}
GET	/api/users/conversations
PUT	/api/users/avatar
DELETE	/api/users/avatar

Figure 3.7: Users API

Orders

GET	/api/orders
POST	/api/orders
GET	/api/orders/{id}
DELETE	/api/orders/{id}
POST	/api/orders/join-group/{orderId}

Figure 3.8: Orders API

Products

GET	/api/products
POST	/api/products
GET	/api/products/{id}
PATCH	/api/products/{id}
DELETE	/api/products/{id}
GET	/api/products/store/{storeId}
GET	/api/products/images/{productId}
POST	/api/products/image
GET	/api/products/image/{ownerId}/{name}
DELETE	/api/products/image/{imageId}

Figure 3.9: Products API

Product_Main_Categories	
GET	/api/product_main_categories
POST	/api/product_main_categories
GET	/api/product_main_categories/{id}
PATCH	/api/product_main_categories/{id}
DELETE	/api/product_main_categories/{id}

Figure 3.10: Product_Main_Categories API

Product_Categories	
GET	/api/product_categories
POST	/api/product_categories
GET	/api/product_categories/{id}
PATCH	/api/product_categories/{id}
DELETE	/api/product_categories/{id}

Figure 3.11: Product_Categories API

Feedbacks	
GET	/api/feedbacks/product/{productId}
POST	/api/feedbacks
PATCH	/api/feedbacks/{id}
DELETE	/api/feedbacks/{id}

Figure 3.12: Feedbacks API

Conversations	
POST	/api/conversations/upload-media
GET	/api/conversations/media/{conversationId}/{name}

Figure 3.13: Conversation API

3.4.3 Token

Tokens were implemented as part of the authentication and authorization system to ensure secure access to protected resources. JWT were used due to their compact nature and ability to carry user information securely. Key features of the token implementation include:

- **Authentication:** Tokens were generated during login and validated for every subsequent request to ensure user identity.
- **Authorization:** Embedded claims within the token determined user roles and access permissions for various resources.
- **Statelessness:** Tokens enabled a stateless authentication system, eliminating the need to store session data on the server.
- **Security:** Tokens were signed using a secret key to prevent tampering and were configured to expire after a set duration to minimize risks.

3.4.4 Security Measures

- **Cross-Origin Resource Sharing (CORS):** The system enforces CORS policies to regulate how resources are accessed from different origins. By defining allowed origins, methods,

and headers, CORS prevents unauthorized cross-origin requests while ensuring secure communication between the frontend and backend. This helps mitigate security risks such as Cross-Site Request Forgery (CSRF) and unauthorized API access.

- **Password Hashing with Bcrypt:** The system uses BCrypt hashing, which includes salting and makes it resistant to brute-force and rainbow table attacks. Each password is hashed before storage, and during authentication, the password is compared to the stored hash.
- **Role-based Access Control (RBAC):** User roles are embedded in the token, enabling the system to enforce different access levels (admin, buyer, seller) for specific resources. The system implements authorization logic to ensure that users can access only the resources they are permitted to.

3.4.5 Data Transfer Object (DTO)

Data Transfer Objects (DTOs) are used in the system to encapsulate and transfer data between different layers, such as between the backend and frontend or between services. DTOs help ensure data consistency, reduce unnecessary data exposure, and improve performance by transferring only the required fields.

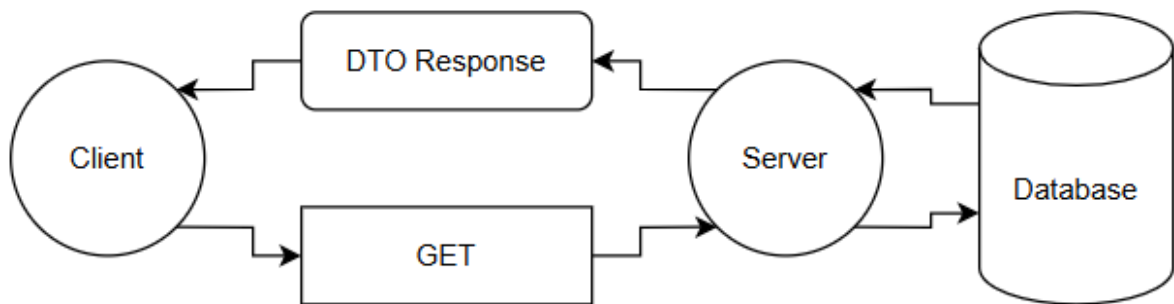


Figure 3.14: Get Request (DTO)

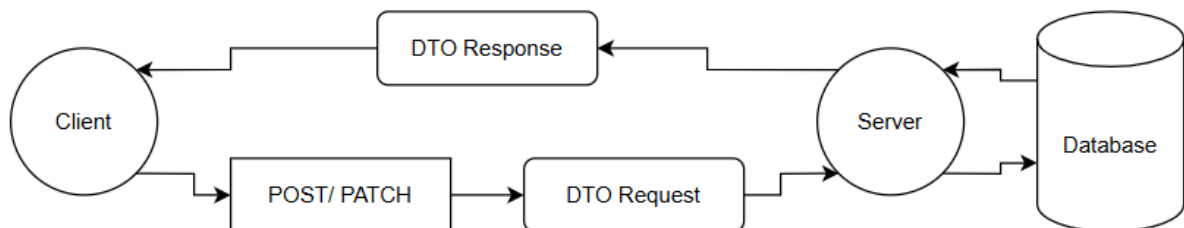


Figure 3.15: Post/ Patch Request (DTO)

- Every piece of data that the server sends to the client is first mapped to a DTO response. This ensures data consistency, enhances security by preventing unnecessary exposure of internal models, and maintains a clear separation between the database structure and the API response format.

- Similarly, when the client submits data to the server, it follows a well-defined DTO request model. The server then maps this DTO request to the corresponding domain model before processing it. This mapping step helps validate incoming data, enforce business rules, and transform data if needed, ensuring a structured and secure data flow within the system.

3.4.6 Email Service

The Email Service is designed to handle all communication with users, including account notifications, order updates, and promotional offers. It ensures that users are promptly informed about important activities such as password resets, purchase confirmations, and shipping details. The service is reliable and efficient, providing a seamless way to keep users engaged and informed.

3.4.8 Auto Scheduled Jobs

A scheduled job named `CheckPaymentTime` is implemented using Quartz, a .NET library for background task scheduling. Its primary function is to monitor overdue shared orders and automatically notify users via email about any pending or canceled sub-orders.

Main Functionality:

- Retrieves all shared orders (*IsShare == true*) and expired *DueTimePayment*.
- Fetches related sub-orders for each shared order.
- If a sub-order:
 - Has not been paid (*IsPaid == false*)
 - Is not using Cash on Delivery (*PaymentMethod != COD*)
- ➔ The system sends an email to the corresponding user:
 - If the payment due date has not passed, it sends a payment reminder.
 - If the payment due date has passed, it sends a sub-order cancellation notice.

Execution Flow:

- Run on a scheduled basis using **Quartz**.
- Check for all expired shared orders and processes the relevant sub-orders.
- Generates and sends emails for each relevant sub-order using **MailService**.

3.4.7 Seed Data

Seed data refers to the initial set of data that is inserted into a database to establish a starting point for an application or system. This data is typically used for testing, development, or demonstration purposes, ensuring that the database contains necessary records or predefined values that allow the application to function correctly. Seed data includes sample users, product listings, or configuration settings, and it helps to simulate real-world data scenarios before the system goes live.

3.4.8 Docker

Docker is an open-source platform that enables packaging, shipping, and running applications inside containers - lightweight, isolated environments that make deployment fast and reliable. In this project, Docker is used to containerize the backend application, ensuring consistency across development, staging, and production environments.

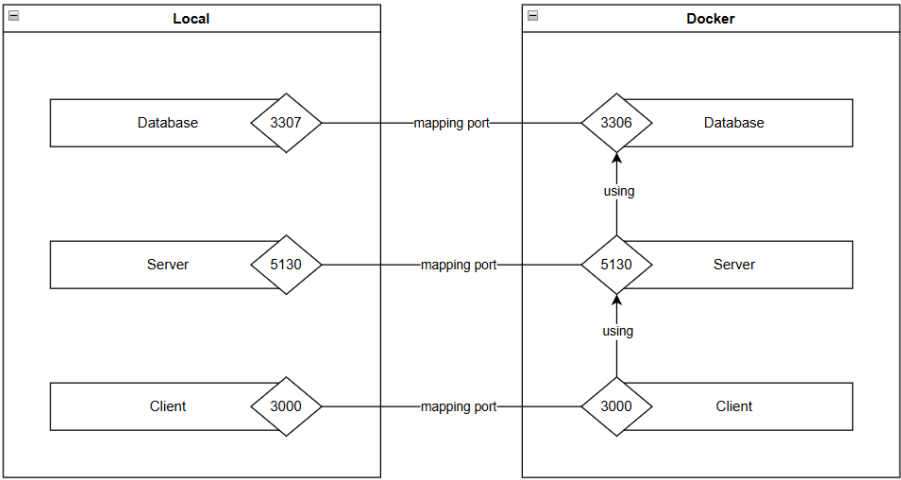


Figure 3.16: Docker Architecture

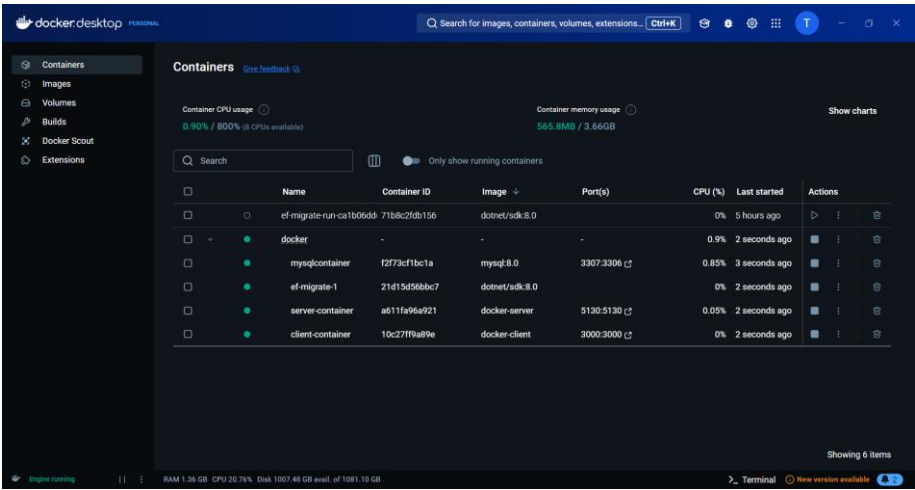


Figure 3.17: Containerize with Docker

Containerize the application using Docker to ensure consistent deployment across environments. Docker enables packaging the app and its dependencies into a lightweight, portable container for efficient development, testing, and production use.

```
Windows PowerShell
(base) PS C:\Users\ScorpioKid> docker tag docker-server trantuanthinh/uni_cap_pro_be:latest
(base) PS C:\Users\ScorpioKid> docker push trantuanthinh/uni_cap_pro_be:latest
The push refers to repository [docker.io/trantuanthinh/uni_cap_pro_be]
79714b2f18d5: Pushed
68871611384b: Pushed
152887e07c59: Pushed
6a8895a0a52: Pushed
54479b024795: Pushed
83fc1d46a296: Pushed
61320b01ae5e: Pushed
8822984bc3f7: Pushed
af4af780ef54: Pushed
a29a682e2ed6: Pushed
latest: digest: sha256:58e45118db76d2e4c7809f38d814ef9224d2662343927a53a7e616c908323dbd size: 856
(base) PS C:\Users\ScorpioKid> docker tag docker-client trantuanthinh/uni_cap_pro_fe:latest
(base) PS C:\Users\ScorpioKid> docker push trantuanthinh/uni_cap_pro_fe:latest
The push refers to repository [docker.io/trantuanthinh/uni_cap_pro_fe]
fe87684b10b0: Pushed
c2d6f8ad9d5ff: Pushed
65b9c193eb7: Pushed
3b83b18dfe1: Pushed
2af21911ea82: Pushed
72a934c64b0: Pushed
cb37fe5b9a8a1: Pushed
19526ecf8c5: Pushed
6f467b68de64: Pushed
latest: digest: sha256:786f7bfa693cae878194d187c56498ab1220f8cc52ff5ecd8a9e7d9ab0958a7f size: 856
(base) PS C:\Users\ScorpioKid>
```

Figure 3.18: Publish to DockerHub

To facilitate application deployment and ensure portability of the development environment, the backend project was containerized as a Docker image.

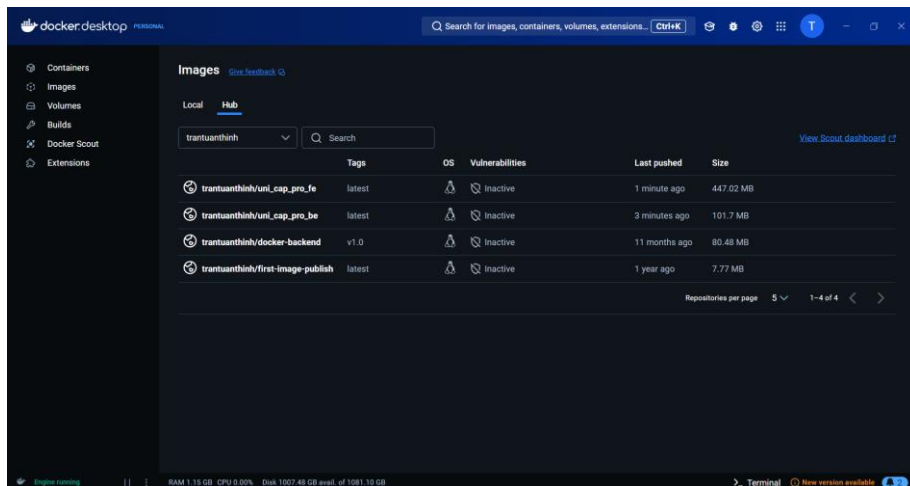


Figure 3.19: DockerHub Published Images

Those images were then pushed to Docker Hub — a popular service for storing and distributing Docker images.

3.5 Front-end Development

Here are some examples of the system's User Interfaces (UI), which showcase the design and layout of key pages and components in the application. These interfaces include elements such as the navigation bar, product listings, user registration, and login forms, all designed to provide an intuitive and seamless user experience.

3.5.1 Banner

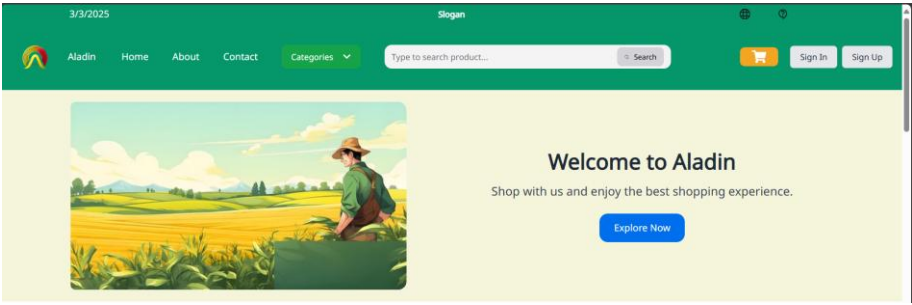


Figure 3.20: Banner

A **Banner** is a prominent visual section, usually located at the top of a webpage, designed to grab users' attention. It often includes images, text, buttons, or animations to convey key messages, promotions, or branding. Website banners can be static or dynamic, adapting to user interactions or displaying rotating content to enhance engagement.

3.5.2 Auth Page

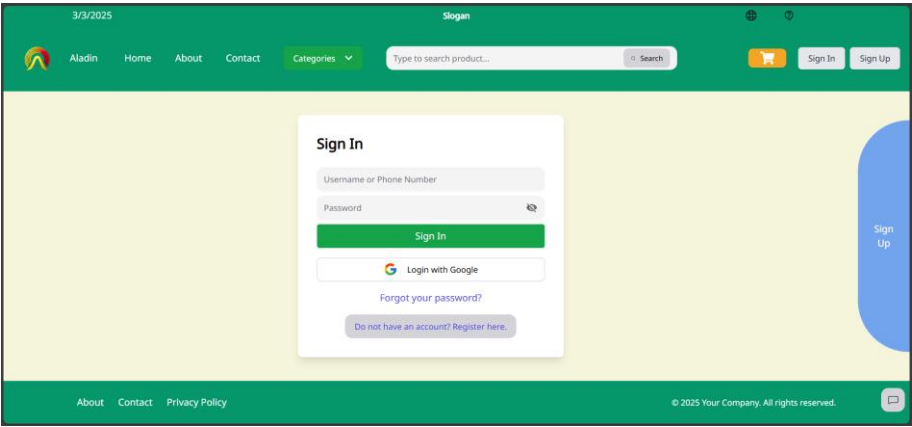


Figure 3.21: Sign In

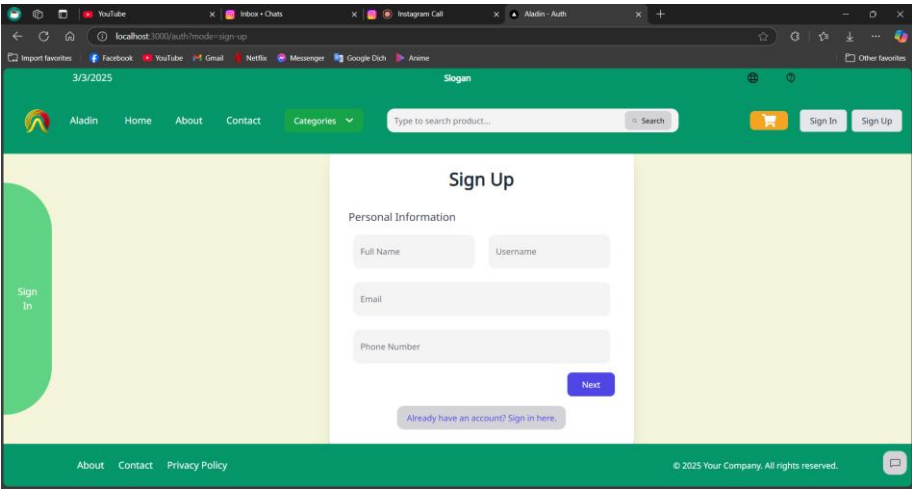


Figure 3.22: Sign Up

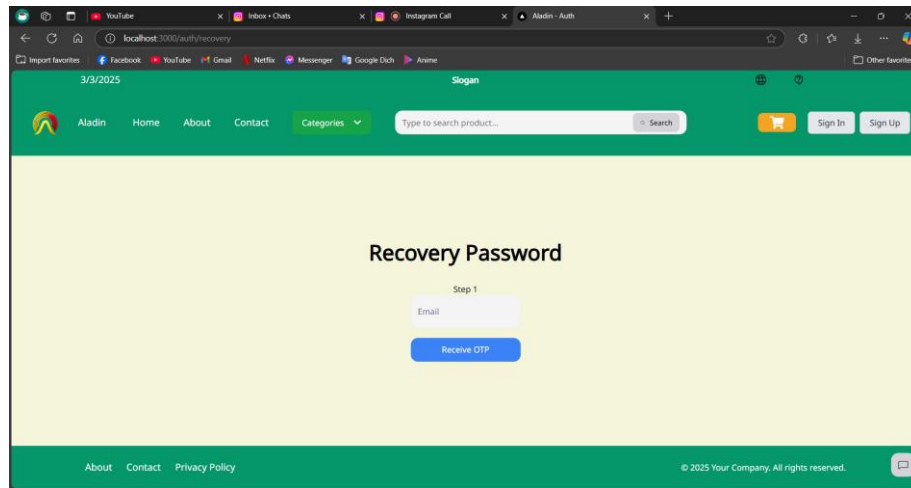


Figure 3.23: Recovery

The **Auth Page** allows users to **sign in** to their existing accounts securely. It also offers the option to **sign up** for new accounts, providing a simple registration process. Additionally, users can easily **recover** their passwords if they forget them, ensuring quick access to their accounts.

3.5.3 Contact Page

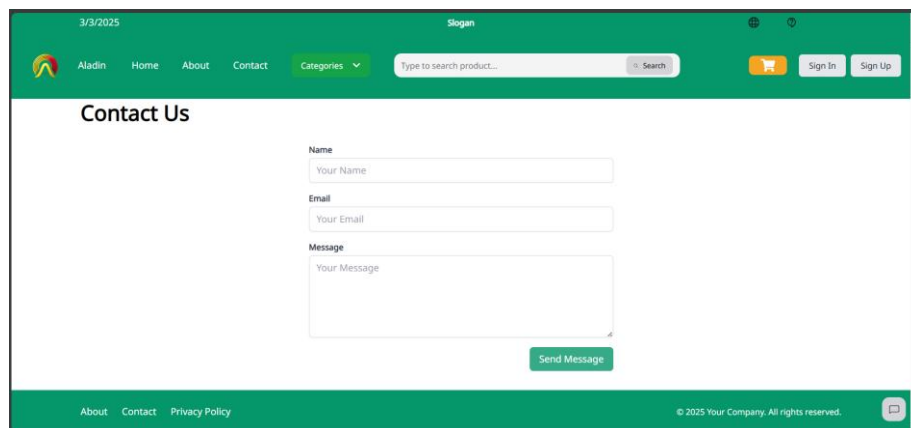


Figure 3.24:Contact

The **Contact Page** enables users to get in touch with the support team for any inquiries or assistance. It provides a simple form to submit questions, feedback, or requests. Users can also find additional contact information, such as email addresses and phone numbers, for direct communication.

3.5.4 Home Page

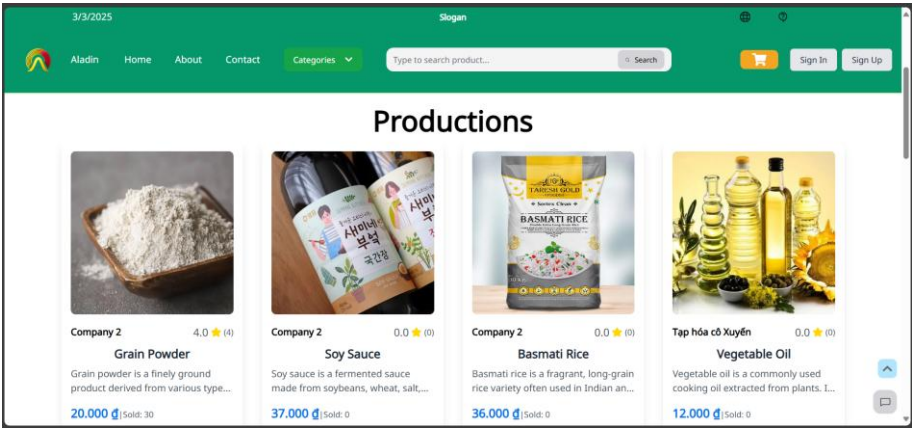


Figure 3.25: Home

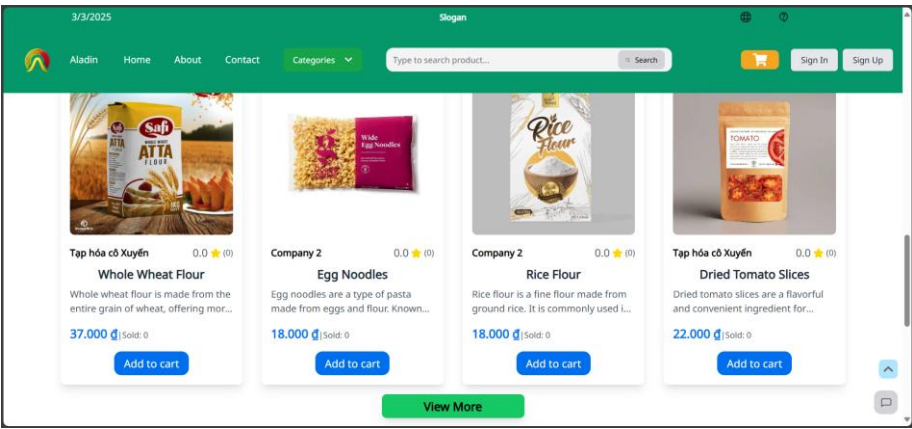


Figure 3.26: Home (with View More Button)

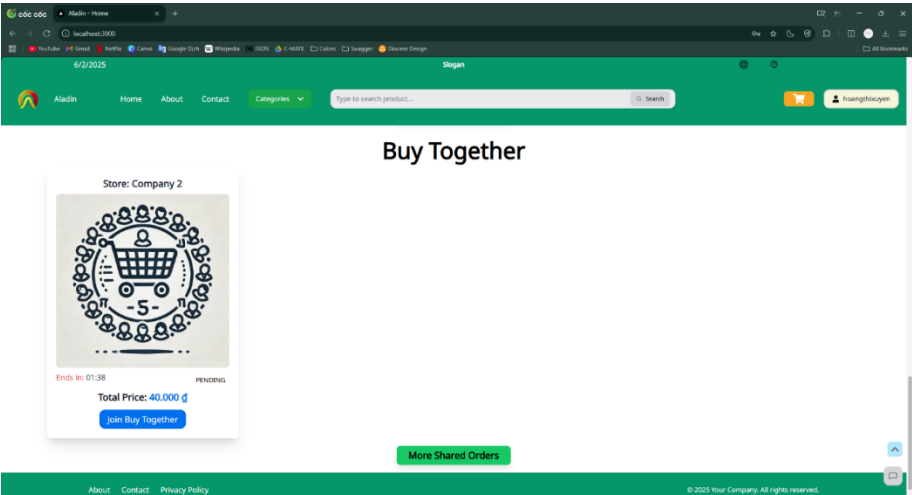


Figure 3.27: Home (Buy Together)

The **Home Page** serves as the main entry point, featuring a list of products and group orders for easy browsing. Users can explore the items and orders in detail, with a 'View More' button for expanded information. This layout ensures a user-friendly experience, allowing visitors to quickly navigate through the available offerings.

3.5.5 Profile

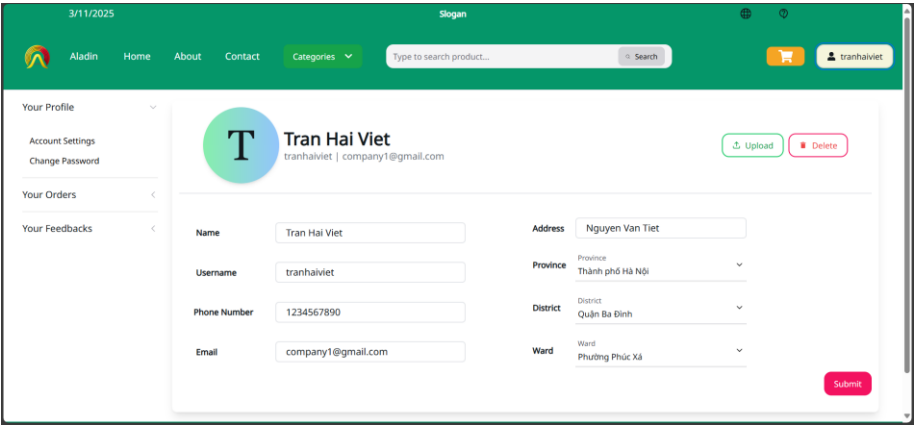


Figure 3.28: Personal Information

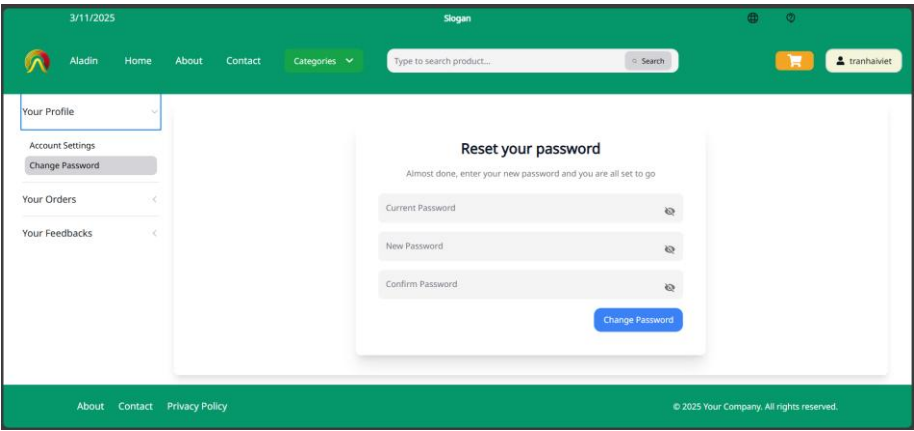


Figure 3.29: Reset Password

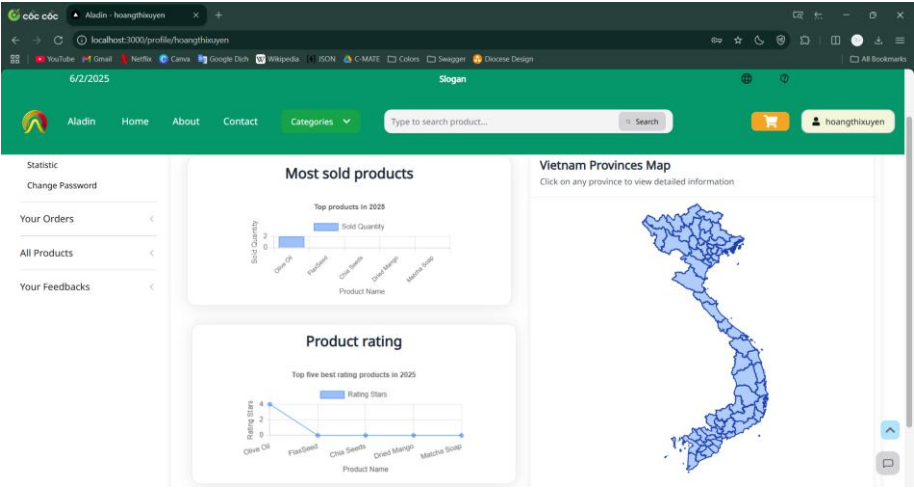


Figure 3.30: Chart/ Map

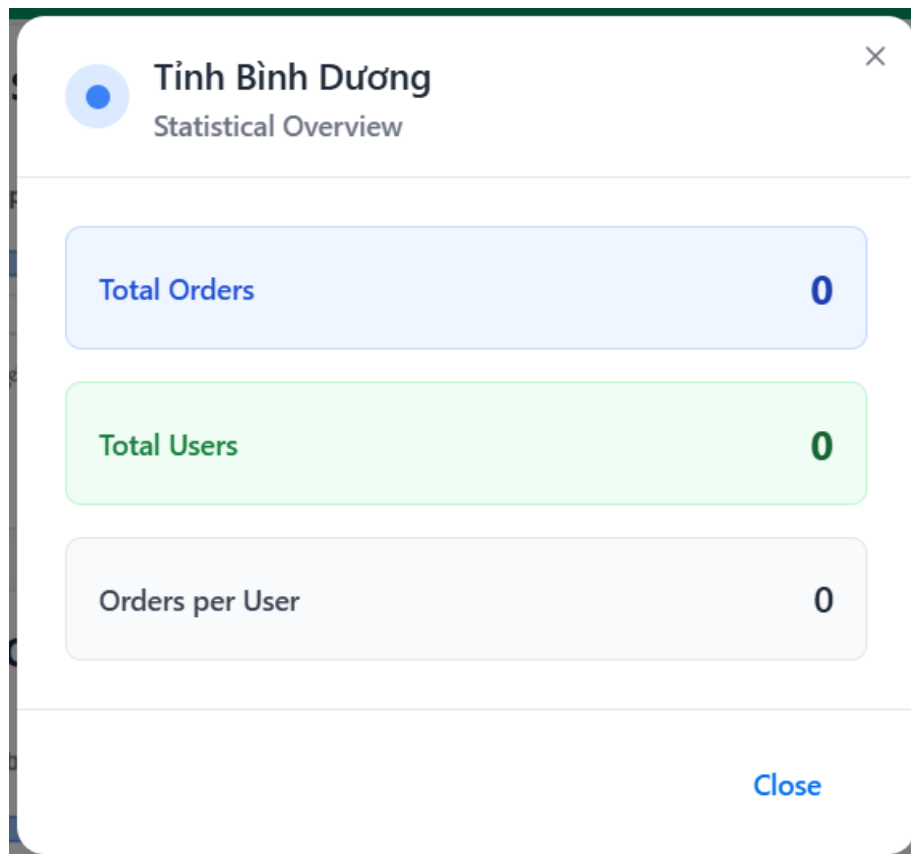


Figure 3.31: Province Detail

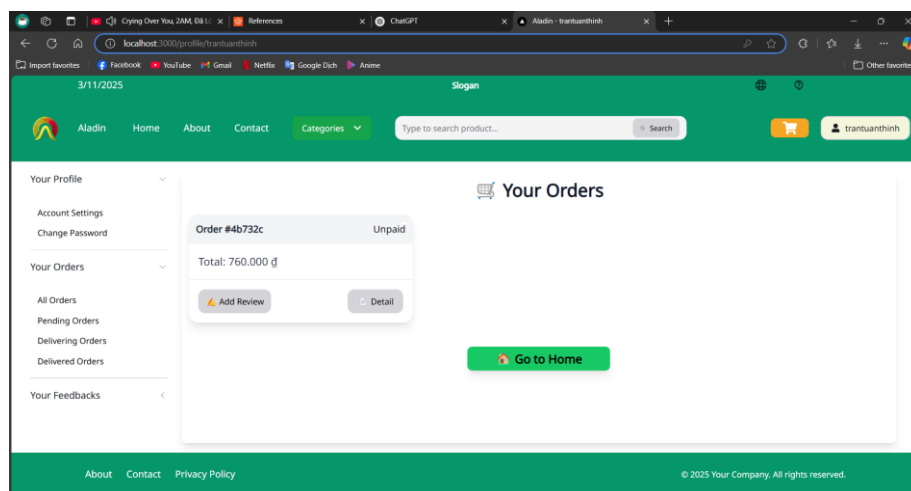


Figure 3.32: Personal History Order

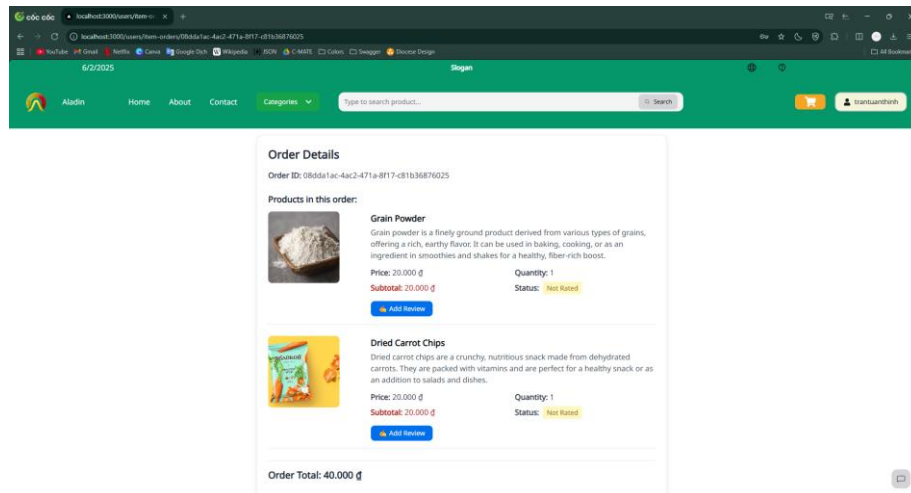


Figure 3.33: Feedback Order

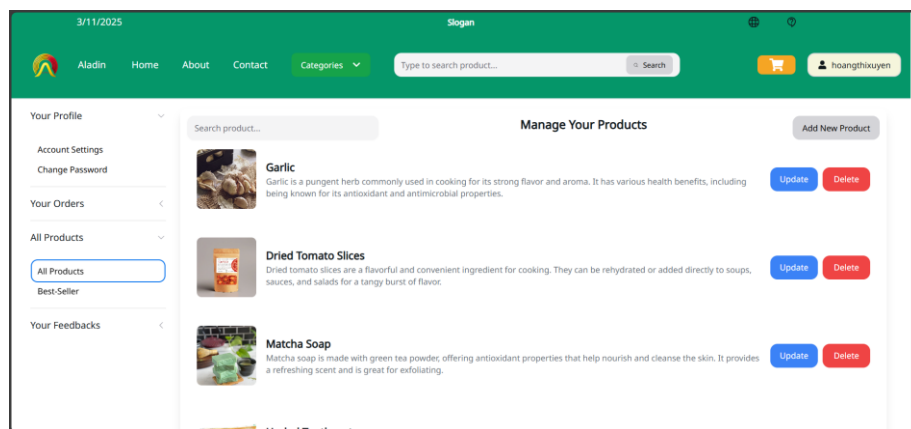


Figure 3.34: Personal Product for Sell

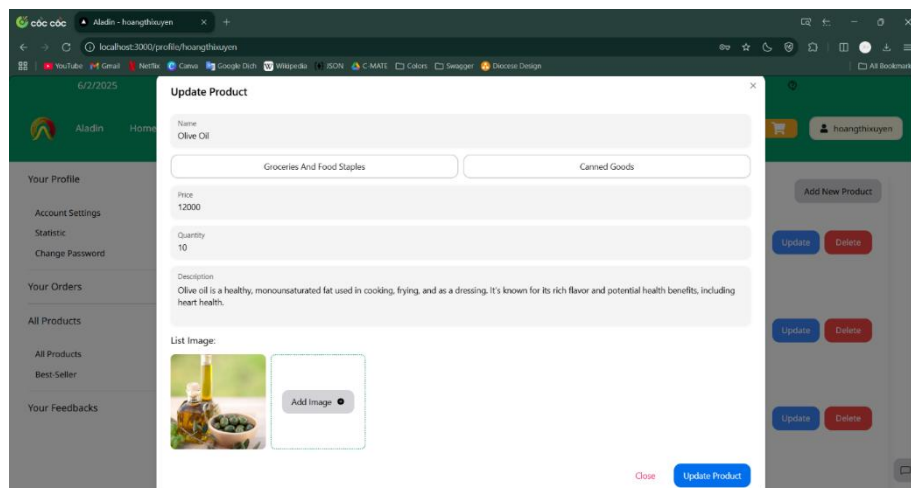


Figure 3.35: Add/ Edit Product

The **Profile Page** allows users to view and update their personal information, ensuring their account details are always up to date. It includes options to reset the password for added security, as well as a section to view order history. For sellers, the page also displays a list of products they have listed, making it easy to manage and track their offerings.

3.5.6 Product Page

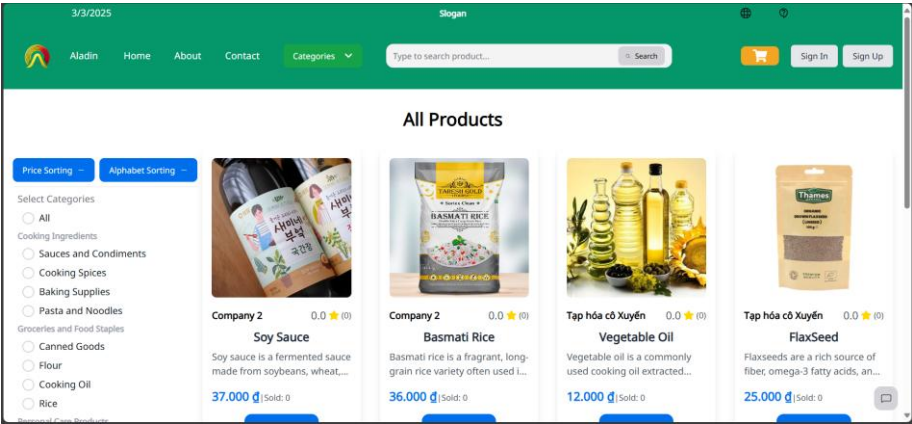


Figure 3.36: Product

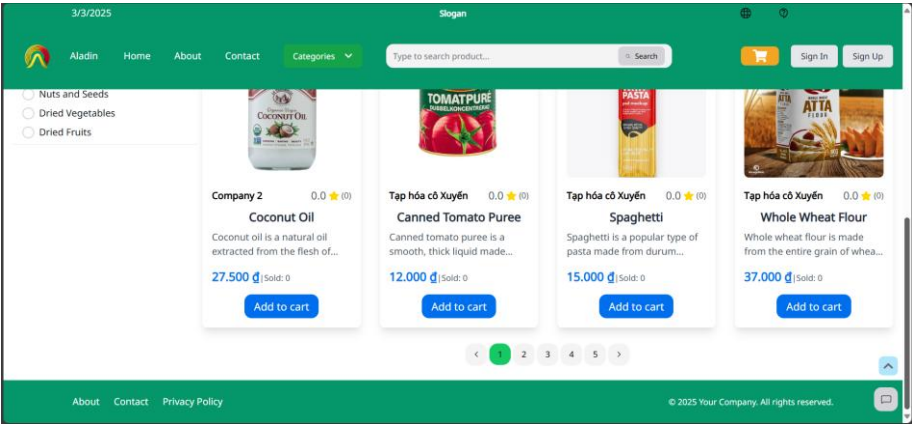


Figure 3.37: Product (with Pagination)

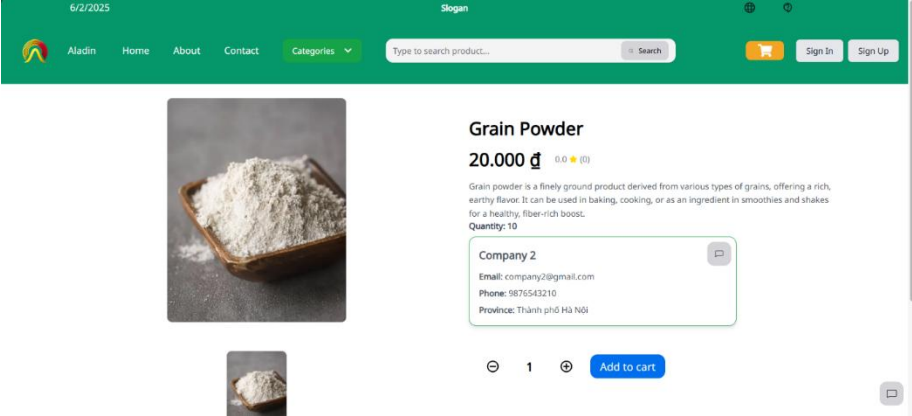


Figure 3.38: Product Detail

The **Product Page** provides detailed information about each product, including descriptions, pricing, and images. Users can easily add items to their cart, view product specifications, and check availability. Additionally, related products may be recommended, offering a seamless shopping experience.

3.5.7 Cart Page

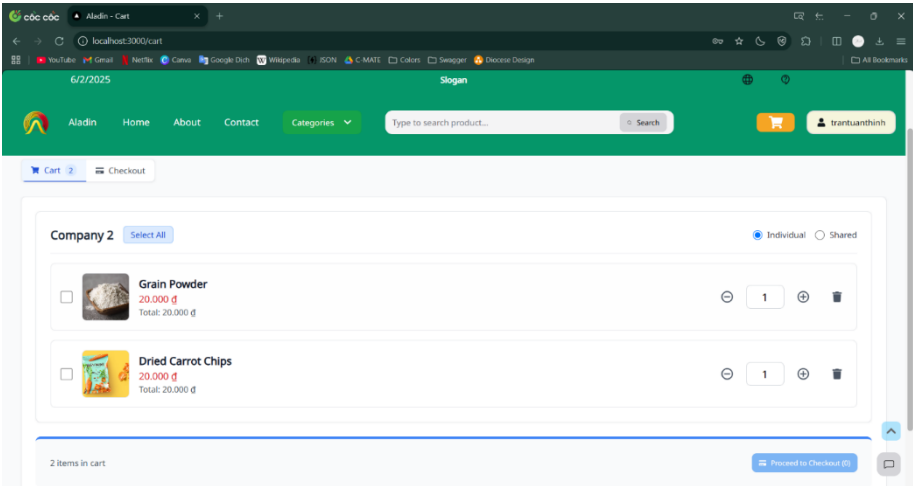


Figure 3.39: Cart

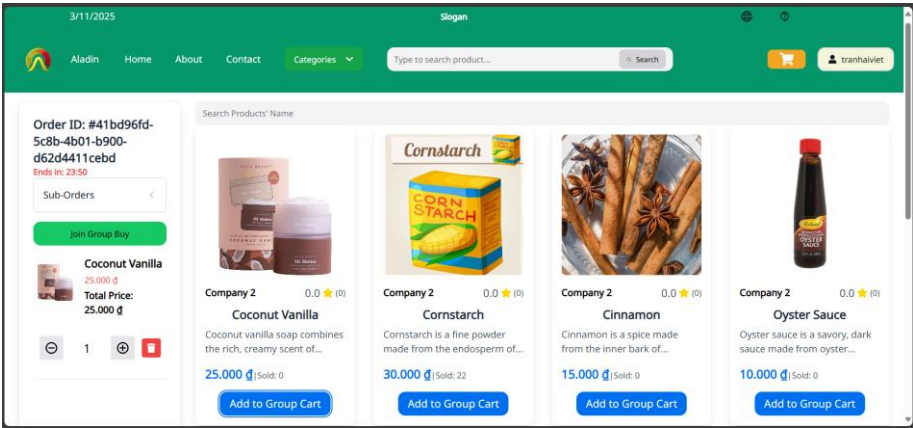


Figure 3.40: Group Buy

The **Cart Page** allows users to review and manage their selected items before checkout. It provides options to add new items to the cart or join an existing group order, making it easy to combine purchases with others. Users can also modify quantities, remove items, and proceed to checkout with ease.

3.5.8 Additional Features



Figure 3.41: Top-up Button



Figure 3.42: Search Product Bar

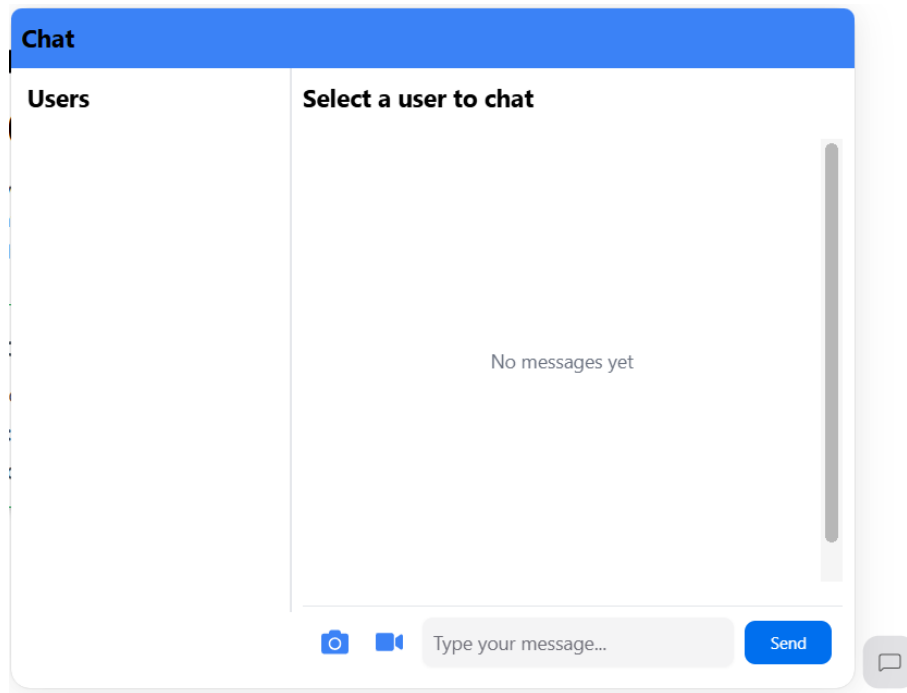


Figure 3.43: ChatBox

The **Additional Features** section includes a top-up button, allowing users to easily add funds to their account. It also features a product search bar for quickly finding specific items by entering keywords, and an integrated chatbox that provides real-time customer support, enhancing the overall shopping experience.

Chapter 4. Conclusion

4.1 Overall Project Summary

In conclusion, the MarketHub project addresses the needs of businesses and agricultural product sellers by providing a platform that enhances purchasing efficiency through a group-buying model. The implementation of technologies like Next.js, ASP.NET, MySQL, and Docker ensures a scalable, user-friendly, and secure system. By combining advanced technical solutions with a clear focus on user needs, this project lays the foundation for future developments in the agricultural e-commerce sector.

4.2 Technological and Methodological Impact

The MarketHub project demonstrates the effective use of modern web technologies, such as Next.js for dynamic rendering and ASP.NET for robust backend development. The use of RESTful APIs ensures seamless communication between the frontend and backend, enabling efficient data management. The integration of containerization with Docker allows for a streamlined development environment. This project has not only expanded my technical knowledge but has also given me hands-on experience in applying these technologies in a real-world context.

4.3 Future Improvements and Scalability

Although the MarketHub platform is functional and supports core features such as group-buying, and product management, there is room for future enhancement. Improving the user interface, integrating online payment, or AI-based recommendations, and enhancing real-time order tracking are potential areas for further development. These improvements will ensure the platform's scalability and adaptability as user demand grows, keeping it competitive in the fast-evolving e-commerce landscape.

4.4 Business and Societal Impact

MarketHub's business model is closely aligned with Goal 10: Reduced Inequalities of the United Nations Sustainable Development Goals (SDGs), which focuses on addressing and minimizing disparities within and between countries:

- **Fairer Pricing and Improved Access:** By offering a group-buying feature, MarketHub enables M-a-P shops, which are often small or local businesses, to pool their purchases and secure products at bulk prices, which they would not have access to individually. This helps reduce the price disparities between large enterprises and smaller businesses, ensuring fairer access to FMCG agricultural products.

- **Reducing Economic Inequality:** The platform's elimination of middlemen reduces additional costs that typically burden smaller businesses and consumers. By creating a direct and transparent connection between distributors and M-a-P shops, MarketHub ensures that profits are fairly distributed, allowing small businesses to grow and thrive. This contributes to reducing economic inequality, particularly in rural or underserved areas.
- **Support for Local and Small-Scale Producers:** By simplifying the supply chain, the platform allows local farmers and small-scale agricultural sellers to reach a broader market, increasing their market share and income. This empowers them economically, reducing the inequality between large agricultural producers and smaller, local suppliers.

As the platform continues to grow, it can serve as a model for other industries looking to optimize supply chain operations and foster sustainable business practices. By empowering M-a-P shops with greater purchasing power through group-buying, businesses can benefit from bulk discounts, while distributors can manage their inventories more effectively, reducing storage and handling costs.

As a result, M-a-P Shops and customers will enjoy better prices when purchasing products, while distributors' profits remain positive due to the elimination of unnecessary fees and logistics inefficiencies. The model ensures a win-win situation for all stakeholders, fostering a more resilient and equitable agricultural market. With continuous development, MarketHub has the potential to revolutionize how FMCG agricultural products are sourced, distributed, and sold in the market.

Chapter 5. Reflection and Future Enhancements

5.1 Overview

This chapter reflects on the key learnings, challenges, and experiences gained during the development of the project. It also explores the potential for future enhancements and improvements, aiming to optimize the system and expand its functionality to meet evolving user needs and market demands.

5.2 Key Learnings

Throughout the project, several important lessons were learned:

- **Technical Skills Development:** working with modern technologies such as **Next.js**, **ASP.NET Core**, and **MySQL**, gaining practical experience in full-stack development. Key tasks like front-end and back-end integration, implementing secure authentication, and managing data interactions helped deepen our understanding of real-world system architecture. Additionally, we learned the fundamentals of **Docker**, which introduced us to containerization and the importance of consistent, isolated environments for development and deployment. This experience provided valuable insights into the complete software development lifecycle—from planning and implementation to testing and deployment.
- **Problem-Solving and Adaptability:** Facing and resolving technical challenges sharpened problem-solving skills. Issues like performance optimization, handling large data volumes, and troubleshooting integrations required creative, adaptive solutions to keep the system reliable and scalable.
- **Project Management:** Using Agile methodologies helped in maintaining focus on deliverables and meeting deadlines. This included task prioritization, iterative development, and regular progress assessments. Working with a version control system (Git) helped ensure that development was streamlined, and changes were tracked efficiently.

5.3 Challenge Faced

While the project achieved its objectives, several challenges emerged that provided valuable learning experiences:

- **MoMo Payment Integration:** Integrating MoMo as a payment gateway presented significant difficulties, especially in handling secure payment flows and verifying transaction responses. Ensuring proper redirection, maintaining consistent order status updates, and handling edge cases like payment cancellations or timeouts required careful implementation and testing. Debugging API callbacks and simulating real transaction scenarios were key to resolving these issues.

- **End-to-End Encryption:** Implementing end-to-end encryption for user communication and sensitive data posed both technical and architectural challenges. Ensuring data security while maintaining performance required a deeper understanding of encryption protocols and secure key exchange mechanisms. Additionally, integrating encryption without breaking functionality—especially in chat features or data transfers—necessitated thorough planning, testing, and optimization.
- **Image Storage with Docker:** Managing image storage within Docker containers was challenging due to container isolation and ephemeral file systems. Ensuring that user-uploaded images were persistently stored outside the container required configuring volume mounts properly. Balancing ease of deployment with data persistence and backup strategies demanded additional attention and experimentation.

5.4 Future Enhancements

To further improve the platform's functionality, performance, and user experience, several enhancements are planned for future development. These include:

- **Mobile Application Support:** Developing native or cross-platform mobile applications to improve accessibility and reach a wider user base.
- **Advanced Recommendation System:** Integrating machine learning models to suggest relevant products based on user behavior, preferences, and purchase history.
- **Real-Time Order Tracking:** Implementing real-time tracking of shipments to increase transparency and customer trust.
- **Improved Security:** Enhancing encryption mechanisms, adding two-factor authentication (2FA), and performing regular vulnerability assessments.
- **Integration with Logistics and Payment Providers:** Adding more shipping partners and supporting additional payment methods beyond MoMo to offer greater flexibility.
- **Automated Notifications:** Enabling email/SMS/push notifications for order updates, promotions, and customer feedback.
- **Admin Analytics Dashboard:** Building a more comprehensive admin dashboard for monitoring performance, sales trends, and user engagement in real time.
- **Deployment with Docker:** Containerizing the application using Docker to ensure consistent environments, simplify deployment processes, and improve scalability and portability.



Đại Học Quốc Tế Miền Đông
Eastern International University

School of Computing and Information Technology Software Engineering

Tran Hai Viet - 2031200032 · Tran Tuan Thinh - 2031200036

Supervisor: Mr. Tran Van Tai

Capstone Project

MARKETHUB E-COMMERCE PLATFORM

A Group-Buying E-commerce Platform for FMCG Agricultural Products

General Idea

A group-buying platform for FMCG agriculture that connects distributors with small retailers, enabling collective purchasing to reduce costs, improve transparency, and simplify bulk buying.

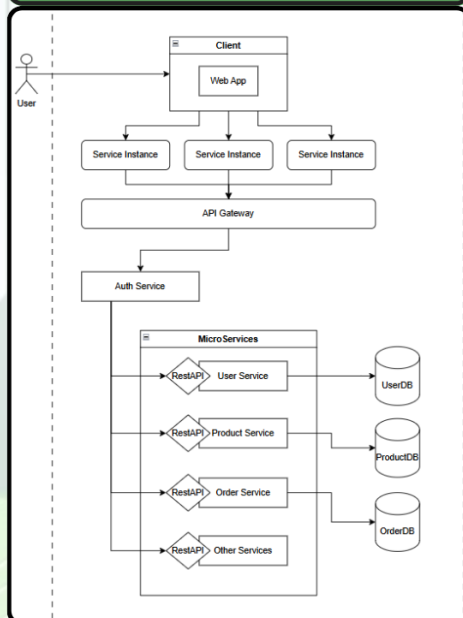
Unique Value

Brings GroupBuy to FMCG agriculture, empowering small retailers - unlike Amazon, ShopeeFood, or GrabFood,

Key Features

GroupBuy Model · Real-time Demand · Visual Dashboard

System Design



Current Model

Factory → Distributors → **Speculators**
→ Mom and Pop Shops → Customers

Problems:

- High prices via speculators
- Overstock from weak demand insight
- Counterfeit goods from poor sourcing
- Inefficient logistics & communication

Solution Model

Factory → Distributors → **MarketHub**
→ Mom and Pop Shops → Customers

Outcomes:

- Group orders save costs
- Live demand matching
- Cut middlemen → fewer counterfeits
- Transparent, local-driven model

Tech: Next.js · ASP.NET Core · MySQL · Docker
Developed by: Tran Tuan Thinh · Tran Hai Viet
Supervisor: Mr. Tran Van Tai

Made with guidance from EIU Faculty

Capstone Project | EIU 2025

Chapter 6. References

- [1] S. Ratliff, “Docker: Accelerated Container Application Development,” *Docker*, Jan. 23, 2025. <https://www.docker.com/>
- [2] “Docker Hub Container Image Library | App Containerization.” <https://hub.docker.com/>
- [3] BillWagner, “C# Guide - .NET managed language,” *Microsoft Learn*. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [4] Hero_Ui, “HeroUI (Previously NextUI) - Beautiful, fast and modern React UI Library,” *HeroUI (Previously NextUI)*. <https://www.heroui.com/>
- [5] R. Vicente, “HTTP cats,” *HTTP Status Cats API*. <https://http.cat/>
- [6] “Git.” <https://git-scm.com/>
- [7] “Lodash.” <https://lodash.com/>
- [8] “MDN Web Docs,” *MDN Web Docs*. <https://developer.mozilla.org/en-US/>
- [9] “Moment.js | Home.” <https://momentjs.com/>
- [10] “Next.Js by Vercel - the React framework.” <https://nextjs.org/>
- [11] “React.” <https://react.dev/>
- [12] “Redux Toolkit | Redux Toolkit.” <https://redux-toolkit.js.org/>
- [13] Soma, “19 Microservices Patterns for system Design Interviews,” *DEV Community*, Aug. 01, 2024. <https://dev.to/somadevtoo/19-microservices-patterns-for-system-design-interviews-3o39/>
- [14] “Emil Kowalski,” *Emil Kowalski*. <https://sonner.emilkowal.ski/>
- [15] “Ideal modeling & diagramming tool for agile team collaboration.” <https://www.visual-paradigm.com/>